

**MATEMATICKO-FYZIKÁLNÍ
FAKULTA**
Univerzita Karlova

BAKALÁŘSKÁ PRÁCE

Adam Šupej

Návrh a implementácia cross-platformovej mobilnej aplikácie pre kontrolu prístupu

Katedra softwarového inženýrství

Vedoucí bakalářské práce: RNDr. Filip Zavoral, Ph.D.

Studijní program: Informatika (B0613A140006)

Praha 2026

Prohlašuji, že jsem tuto bakalářskou práci vypracoval(a) samostatně a výhradně s použitím citovaných pramenů, literatury a dalších odborných zdrojů. Beru na vědomí, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., autorského zákona v platném znění, zejména skutečnost, že Univerzita Karlova má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle §60 odst. 1 autorského zákona.

V dne

Podpis autora

Rád by som poďakoval môjmu vedúcemu pánovi RNDr. Filipovi Zavoralovi, Ph.D. za jeho odborné vedenie, cenné rady a ochotu, ktorú mi venoval počas celého procesu vypracovania tejto bakalárskej práce.

Název práce: Návrh a implementácia cross-platformovej mobilnej aplikácie pre kontrolu prístupu

Autor: Adam Šupej

Katedra: Katedra softwarového inžénýrství

Vedoucí bakalářské práce: RNDr. Filip Zavoral, Ph.D., Katedra softwarového inžénýrství

Abstrakt: Bakalárska práca sa zaoberá návrhom a implementáciou cross-platformovej mobilnej aplikácie kontroly prístupu do budov. Výsledný prototyp umožňuje centralizovanú správu prístupových práv v hierarchickej štruktúre organizácie, digitálne odomykanie zámkov a priradenie oprávnení používateľom. Práca analyzuje požiadavky na moderné systémy kontroly prístupu, navrhuje architektúru riešenia a implementuje funkčný prototyp s dôrazom na bezpečnosť a intuitívnosť používania. Výsledkom je mobilná aplikácia kompatibilná s iOS a Android platformami, ktorá predstavuje modernú alternatívu k tradičným mechanickým kľúčom a kartovým systémom.

Klíčová slova: mobilná aplikácia, správa prístupu, flutter, nfc, esp32.

Title: Design and implementation of a cross-platform mobile application for access control

Author: Adam Šupej

Department: Department of Software Engineering

Supervisor: RNDr. Filip Zavoral, Ph.D., Department of Software Engineering

Abstract: This bachelor's thesis focuses on the design and implementation of a cross-platform mobile access control application for buildings. The resulting prototype enables centralized management of access rights within a hierarchical organizational structure, digital lock unlocking, and user permission assignment. The thesis analyzes requirements for modern access control systems, designs the solution architecture, and implements a functional prototype with emphasis on security and user-friendliness. The result is a mobile application compatible with iOS and Android platforms, representing a modern alternative to traditional mechanical keys and card-based systems.

Keywords: mobile application, access control, flutter, nfc, esp32.

Obsah

Úvod	8
1 Analýza systému	9
1.1 Existujúce riešenia	9
1.1.1 Kartové systémy na báze RFID	9
1.1.2 Bezdrôtové riešenia BLE	10
1.1.3 Mobilné peňaženky	10
1.1.4 Komerčne integrované platformy	10
1.1.5 Pozícia navrhovaného systému	11
1.2 Špecifikácia požiadaviek	11
1.2.1 Funkčné požiadavky	11
1.2.2 Nefunkčné požiadavky	11
1.3 Analýza fyzického hardvéru	13
1.3.1 Požiadavky na hardvér	13
1.3.2 Porovnanie dostupných platforiem	14
1.3.3 Výber vývojovej dosky	14
1.3.4 NFC modul	15
1.3.5 Hardvérová zostava ako celok	15
1.4 Analýza softvérového stacku	15
1.4.1 Mobilná aplikácia	15
1.4.2 Serverová vrstva	16
1.4.3 Cloudová IoT vrstva	17
1.4.4 Firmvér IoT zariadenia	18
1.4.5 Zhrnutie softvérového stacku	18
1.5 Koncepty hardvérového stacku a NFC komunikácie	18
1.5.1 Mobilný telefón ako emulovaný NFC tag	19
1.5.2 ESP32 v kombinácii s NFC modulom PN532	20
1.5.3 Využitie dynamického NFC tagu	21
1.5.4 Zhrnutie iteratívneho vývoja	22
2 Návrh systému a architektúry	23
2.1 Architektúra systému a topológia	23
2.1.1 Architektúra komponentov	23
2.1.2 IPv4 a IPv6 kompatibilita	24
2.1.3 Centralizovaná autorizácia	25
2.2 Návrh databázového modelu	25
2.2.1 Princípy návrhu	25
2.2.2 Organizačná štruktúra	27
2.2.3 Entity používateľov	27
2.2.4 Entita prístupových kľúčov	27
2.2.5 Spojové tabuľky	28
2.2.6 Enumeračné typy	28
2.2.7 Prepojenie s autentifikačnou vrstvou	28
2.3 Komunikačný tok odomykacieho procesu	28
2.3.1 Celkový tok	29

2.3.2	Pohľad mobilnej aplikácie	29
2.3.3	Pohľad serverovej vrstvy	29
2.3.4	Pohľad IoT zariadenia	32
2.4	Bezpečnostný návrh	32
2.4.1	Model hrozieb	32
2.4.2	Transportná bezpečnosť	34
2.4.3	Autentifikácia používateľa	34
2.4.4	Autorizácia	34
2.4.5	Autentifikácia komunikácie medzi servermi	35
2.4.6	Princíp najmenšieho privilégia	35
2.4.7	Bezpečnosť IoT zariadenia	36
3	Implementácia	37
3.1	Implementácia mobilnej aplikácie	37
3.1.1	Použité technológie a kľúčové koncepty	37
3.1.2	Integrácia natívneho kódu	37
3.1.3	Architektúra a správa stavu aplikácie	40
3.1.4	Bezpečné ukladanie používateľských tokenov	42
3.1.5	Správa vývojových prostredí	42
3.1.6	Implementácia používateľského rozhrania	44
3.2	Backendová a cloudová vrstva	48
3.2.1	Cloudflare Worker ako proxy	48
3.2.2	Databáza a Edge Functions	48
3.3	Implementácia hardvéru a IoT komunikácie	50
3.3.1	Komunikácia s dynamickým NFC tagom	50
3.3.2	Integrácia s Azure IoT Hub cez MQTT	50
3.3.3	Bezpečnostné obmedzenia prototypu	51
4	Evaluácia	53
4.1	Testovacia stratégia	53
4.2	Funkčné testovanie	54
4.3	Výkonnostné meranie	57
4.4	Testovanie sieťovej kompatibility	59
4.5	Vyhodnotenie funkčných požiadaviek	61
4.6	Vyhodnotenie nefunkčných požiadaviek	61
4.7	Súhrn evaluácie a výhľad	62
	Záver	64
	Literatúra	65
	Zoznam obrázkov	70
	Zoznam použitých skratiek	71
A	Prílohy	73
A.1	Mobilná aplikácia	73
A.2	Arduino kód	73
A.3	Edge Functions	73

A.4	Supabase schéma	73
A.5	Cloudflare Worker	73

Úvod

Kontrola prístupu k fyzickým priestorom patrí medzi najstaršie úlohy, ktoré ľudstvo riešilo technologickými prostriedkami. Od mechanických zámkov používaných už v staroveku až po moderné elektronické systémy princíp zostáva rovnaký. Cieľom je rozlíšiť oprávnený subjekt od neoprávneného a selektívne mu povoliť, prípadne zamietnuť prechod cez fyzickú bariéru. Menia sa však nosiče identity a spôsoby, akými je táto identita overovaná. V posledných dvoch desaťročiach sa popri tradičných mechanických kľúčoch a kartových systémoch na báze RFID presadzujú riešenia, ktoré ako nosič identity využívajú zariadenia, ktoré má používateľ pri sebe z iných dôvodov. Podľa dát Eurostatu sa v roku 2025 takmer deväť z desiatich používateľov internetu v Európskej únii pripájalo cez mobilné zariadenia, čo z nich robí všadeprítomný osobný nástroj [1]. Tento fakt vytvára prirodzenú motiváciu pre návrh prístupových systémov, ktoré mobilné zariadenie využívajú ako digitálny kľúč.

Prenesenie digitálnej identity z fyzickej karty do mobilnej aplikácie rieši časť problémov kartových systémov. Kľúče sa distribuujú vzdialene, je možné ich revokovať v reálnom čase a časovo obmedziť, pričom fyzický zámok môže byť pripojený do cloudovej infraštruktúry a jeho rozhodovanie centralizované. Táto architektúra však prináša vlastné výzvy, najmä v oblasti zabezpečenia komunikačných kanálov, návrhu oprávňovacieho modelu pre hierarchickú organizáciu a zabezpečenia cross-platformnej kompatibility medzi dominantnými mobilnými platformami iOS a Android.

Cieľom tejto práce je navrhnúť a implementovať funkčný prototyp cross-platformového systému kontroly prístupu, ktorý využíva smartfón ako nosič digitálnej identity a NFC ako kanál pre interakciu so zámkom. Systém pozostáva z piatich komponentov, mobilnej aplikácie, serverovej vrstvy postavenej na platforme BaaS s pomocným proxy komponentom, cloudového IoT prostredia a firmvéru mikrokontroléra zámku. Spojenie komponentov tvorí end-to-end reťazec od načítania NFC tagu po fyzické uvoľnenie zámku. Súčasťou práce je aj dokumentácia iteratívneho procesu, ktorým sa k finálnemu návrhu dospelo. Prototyp prešiel tromi revíziami, z ktorých prvé dve boli zamietnuté pre technické či regulačné obmedzenia.

Text je organizovaný do štyroch kapitol. Kapitola 1 analyzuje doménu kontroly prístupu, existujúce riešenia, špecifikuje funkčné a nefunkčné požiadavky a popisuje tri zvažované koncepty hardvérového stacku. Kapitola 2 predstavuje návrh systémovej architektúry, dátového modelu a bezpečnostnej vrstvy. Kapitola 3 sa venuje implementácii jednotlivých komponentov, mobilnej aplikácie, backendovej a cloudovej vrstvy a firmvéru IoT zariadenia. Kapitola 4 vyhodnocuje prototyp z hľadiska funkčnosti, výkonu a sieťovej kompatibility a porovnáva dosiahnutý stav so špecifikovanými požiadavkami. Záver sumarizuje dosiahnuté výsledky a načrtáva možnosti ďalšieho rozvoja.

1 Analýza systému

Kapitola analyzuje doménu elektronickej kontroly prístupu, existujúce riešenia na trhu, funkčné a nefunkčné požiadavky na navrhovaný systém a zdôvodnenie hardvérových komponentov a softvérových technológií, ktoré tvoria základ architektúry. Záver kapitoly dokumentuje tri iteratívne zvažované koncepty NFC komunikácie.

1.1 Existujúce riešenia

Systémy kontroly prístupu k budovám a jednotlivým priestorom možno rozdeliť do niekoľkých generácií podľa spôsobu, akým reprezentujú identitu používateľa a akou technológiou komunikujú s fyzickým zámkom. Najjednoduchším a historicky najstarším riešením sú mechanické zámky s kovovými kľúčmi, ktoré sú stále rozšírené v rezidenčnej sfére. Ich obmedzenia sú však v prostredí s vyšším počtom oprávnených osôb a častými zmenami zloženia používateľov, ako sú napríklad hotely, administratívne budovy alebo zdieľané pracoviská, dobre známe. Kľúč sa dá jednoducho stratiť alebo skopírovať, jeho platnosť nemožno vzdialene zrušiť a pri kompromitácii je nutná fyzická výmena celej vložky. Elektronické systémy riešia tieto problémy tým, že identita používateľa je reprezentovaná digitálnym tokenom uloženým buď na fyzickom zariadení, či už ide o kartu, alebo kľúčenku, alebo v mobilnom zariadení. Nasledujúci prehľad predstavuje hlavné kategórie riešení, s ktorými navrhovaný prototyp priamo alebo nepriamo konkuruje.

1.1.1 Kartové systémy na báze RFID

Najrozšírenejšou kategóriou elektronickej kontroly prístupu sú kartové systémy, ktoré využívajú technológiu RFID a jej podmnožinu NFC. Dominantnou rodinou produktov je platforma MIFARE spoločnosti NXP Semiconductors. Staršia varianta MIFARE Classic, nasadená celosvetovo v stovkách miliónov inštancií, sa ukázala ako kryptograficky slabá. Praktické útoky na CRYPTO1 predstavili de Koning Gans, Hoepman a Garcia [2] a Courtois [3]. Výsledným faktom je, že obnovenie 48-bitového tajného kľúča je možné v priebehu sekúnd až minút na bežnom počítači.

Novšie generácie rodiny MIFARE tento nedostatok riešia. Varianty DESFire EV2 a EV3 využívajú hardvérovú 128-bitovú AES šifru, podporujú obojstrannú autentifikáciu, transakčné MAC, proximity check proti relay útokom a sú certifikované podľa Common Criteria na úroveň EAL5+ pre hardvér integrovaného obvodu [4]. Ich nasadenie v prostredí komerčnej kontroly prístupu je dnes veľmi rozšírené, pričom kompatibilné produkty dodávajú okrem NXP aj integrátori ako HID Global [5].

Hlavnou nevýhodou kartových systémov ako kategórie, bez ohľadu na kryptografickú silu nosiča, je nutnosť fyzickej distribúcie média. Strata karty znamená nutnosť jej fyzickej náhrady, revokácia prístupu vyžaduje buď vrátenie karty, alebo jej pridanie na čiernu listinu v každom jednotlivom zámku. Pri hromadnom nasadení vzniká tiež environmentálny náklad spojený s výrobou a likvidáciou plastových kariet.

1.1.2 Bezdrôtové riešenia BLE

Technológia BLE sa stala základom prvej generácie masovo nasadených mobilných prístupových systémov. Rozšírený dosah umožňuje interakciu bez fyzického priloženia telefónu k zámku, čo je z hľadiska používateľského komfortu výhodné. Medzi komerčne najvýznamnejších dodávateľov patrí spoločnosť Salto Systems s platformou Salto KS, ktorá predstavuje cloudovo spravovaný systém kontroly prístupu, ktorý ako primárny komunikačný kanál medzi mobilným zariadením a zámkom využíva práve BLE [6]. Podobnou architektúrou sa vyznačuje aj rodina produktov ASSA ABLOY Aperio a riešenia skupiny dormakaba.

BLE ako kanál má však viaceré nevýhody v porovnaní s NFC. V scenári, kde zámok trvalo udržiava aktívnu rádiovú reláciu, je energeticky náročnejší ako pasívny NFC tag. Vyžaduje aj párovanie a komplexnejšie protokoly nadviazania spojenia, čo predlžuje čas odozvy. Zásadným bezpečnostným problémom je náchylnosť na relay útok pre svoj zväčšený dosah, ktorý Staat a kol. [7] prakticky demonštrovali na komerčných systémoch typu *iPhone-as-a-Key* vrátane elektrického zámku Nuki Smart Lock 2.0. Používateľský model je tiež menej intuitívny, kde namiesto jednoznačného fyzického priloženia je nutné sa spoľahnúť na automatickú detekciu, čo vedie k nejednoznačným situáciám pri viacerých dverách v tesnej blízkosti. Niektoré BLE systémy, vrátane Salto KS, preto ponúkajú aj NFC ako alternatívny kanál [8].

1.1.3 Mobilné peňaženky

Najnovšia generácia mobilných prístupových riešení integruje digitálny kľúč priamo do natívnej peňaženky operačného systému. Apple Wallet podporuje niekoľko typov prístupových objektov, ako napríklad kľúče k domu, hotelové kľúče, firemné zamestnanecké karty, študentské identifikácie a kľúče od vozidla. Komunikácia so zámkom prebieha cez NFC a je navrhnutá tak, aby vyžadovala minimálnu interakciu, v niektorých prípadoch funguje bez odomknutia zariadenia a krízovo aj pri vybití batérie. Bezpečnostný model sa opiera o Secure Element integrovaný v zariadení [9, 10].

Pre integrátorov a prevádzkovateľov zariadení má však toto riešenie niekoľko obmedzení. Vstup do programu Apple Wallet Keys vyžaduje absolvovanie biznisovej a technickej certifikácie vrátane overenia kompatibility so špecifickým radom hardvérových čítačiek [11]. Systém je tým pádom uzavretý pre autorizovaných partnerov a výrobcov zámkov, ktorí sa rozhodnú do ekosystému vstúpiť za stanovených podmienok. Pre vývojára, ktorý chce implementovať vlastný prístupový systém na nezávisle dostupnom hardvéri, predstavuje Apple Wallet Keys nedostupnú cestu. Google Wallet ponúka podobnú funkcionality s porovnateľnými obmedzeniami.

1.1.4 Komerčne integrované platformy

Popri uvedených technológiách existuje rad komplexných komerčných platform, ktoré kombinujú hardvér, softvér a služby do jedného integrovaného produktu. Medzi najvýznamnejších globálnych dodávateľov patria dormakaba, ASSA, ABLOY, Allegion a Salto Systems. Tieto riešenia typicky ponúkajú širokú funkcionality, vrátane hierarchickej správy prístupových práv, auditu udalostí

a integrácie na systémy správy budov. Cenovo sú však zamerané predovšetkým na korporátny segment a pre menších nasadzovateľov sú náklady na hardvér a licencie často bariérou. Zároveň väzba na jedného dodávateľa znamená obmedzenú možnosť miešať komponenty rôznych výrobcov.

1.1.5 Pozícia navrhovaného systému

Z prehľadu vyplýva, že kvalitná mobilná kontrola prístupu je dnes dostupná najmä ako uzavretý komerčný produkt, viazaný buď na ekosystém natívnej peňaženky, alebo na hardvérovo-softvérovú platformu jedného dodávateľa. Navrhovaný prototyp predstavuje alternatívnu cestu postavenú výlučne na hardvéri voľne dostupnom v maloobchode a na otvorených alebo verejne dokumentovaných cloudových službách. Smartfón v úlohe NFC čítačky odstraňuje fyzickú distribúciu nosiča, voľba NFC pred BLE udržiava jednoznačnú interakciu pri minimálnej spotrebe na strane zámku a architektúra zostáva nezávislá od programu schvaľovaného výrobcom platformy. Práca neusiluje o priamu náhradu etablovaných komerčných systémov, ale o doloženie tézy, že kľúčové architektonické vlastnosti (vzdialená správa, koncové šifrovanie, hierarchická autorizácia) sú dosiahnuteľné aj nad otvorenou kombináciou všeobecne dostupných stavebných blokov.

1.2 Špecifikácia požiadaviek

Sekcia formalizuje požiadavky na navrhovaný systém. Sú rozdelené do dvoch kategórií podľa štandardu IEEE/ISO/IEC 29148 [12]. Funkčné požiadavky popisujú schopnosti a správanie systému, nefunkčné požiadavky špecifikujú jeho kvalitatívne vlastnosti. Každá požiadavka je označená identifikátorom umožňujúcim odkazovanie pri vyhodnotení jej splnenia.

1.2.1 Funkčné požiadavky

Funkčné požiadavky popisujú konkrétne schopnosti, ktoré musí systém poskytovať. Funkčné požiadavky sú zhrnuté v tabuľke 1.1, pričom dve z nich si vyžadujú vysvetlenie.

FP5, vznik oprávnenia. Oprávnenie odomknúť konkrétne zariadenie vzniká, ak existuje aspoň jedno platné aktívne pridelenie prístupového kľúča používateľovi, pričom pokrýva dané zariadenie a aktuálny čas spadá do časovej platnosti pridelenia. Časová platnosť patrí pri tom samotnému prideleniu, nie kľúču ako takému.

FP7, rozdelenie zodpovedností. Mobilná aplikácia slúži len ako kanál pre čítanie NFC identifikátora a odosielanie autorizačných požiadaviek. Fyzické zariadenie vykonáva príkaz doručený zo servera, ale oprávnenia samo nevyhodnocuje. Toto rozdelenie umožňuje okamžitú revokáciu prístupu bez synchronizácie do koncových zariadení.

1.2.2 Nefunkčné požiadavky

Nefunkčné požiadavky špecifikujú kvalitatívne vlastnosti systému ako celku. Tabuľka 1.2 ich sumarizuje, dve z nich sú podrobnejšie rozpracované v texte.

ID	Požiadavka	Popis
FP1	Autentifikácia používateľa	Prihlásenie e-mailom a heslom, vystavenie prístupového tokenu, vymazanie lokálnych údajov pri odhlásení.
FP2	Hierarchická organizačná štruktúra	CRUD operácie pre štvorúrovňovú hierarchickú spoločnosť → oblasť → budova → zariadenie.
FP3	Správa používateľských účtov	Vytváranie účtov so systémovou rolou, viazanou na spoločnosť, dostupné iba administrátorským rolám.
FP4	Správa prístupových kľúčov	Definovanie pomenovanej množiny zariadení ako abstraktnej entity opakovane priradovanej používateľom.
FP5	RBAC oprávnenia	Priradenie kľúča používateľovi s voliteľnou časovou platnosťou, možnosť predčasného odňatia.
FP6	Odomknutie cez NFC	Načítanie identifikátora z NFC tagu, autorizácia na serveri, vizuálna spätná väzba zámku aj aplikácie.
FP7	Centralizovaná autorizácia	Rozhodovanie výhradne na serverovej vrstve, klient ani zámok oprávnenia nevyhodnocujú.
FP8	Logické vymazanie	Soft delete cez príznak, zachovanie referenčnej integrity historických záznamov.
FP9	Viacjazyčné rozhranie	Slovenčina, čeština, angličtina. Jazyk podľa nastavení operačného systému.
FP10	Vývojové a produkčné prostredie	Konfiguračné prepínanie prostredí bez zmeny zdrojového kódu.

Tabuľka 1.1 Funkčné požiadavky na systém kontroly prístupu.

ID	Požiadavka	Popis
NFP1	Multiplatformnosť	Funkčnosť na iOS aj Android s ekvivalentným používateľským rozhraním.
NFP2	Bezpečnosť komunikácie a údajov	Šifrovaná komunikácia cez TLS [13], JWT autorizácia [14], hardvérovo chránené úložisko tokenov.
NFP3	Čas odozvy	Odomykací tok do 3 sekúnd v bežných sieťových podmienkach.
NFP4	Škálovateľnosť	Architektúra umožňujúca rast počtu zariadení, používateľov a spoločností bez zmien v návrhu.
NFP5	Použiteľnosť	Odomknutie jedným ťuknutím a priložením telefónu, oddelená administratívna sekcia.
NFP6	Udržiavateľnosť	Oddelenie vrstiev (UI, stav, dáta, komunikácia) v zdrojovom kóde mobilnej aplikácie.
NFP7	Sieťová kompatibilita	Funkčnosť aj v sieťach bez podpory IPv6.

Tabuľka 1.2 Nefunkčné požiadavky na systém kontroly prístupu.

NFP3, bežné podmienky. Troj sekundový limit platí pri stabilnom Wi-Fi alebo mobilnom pripojení s latenciou do 200 ms voči serverovej vrstve. Pri zhoršenej konektivite môže čas odozvy rásť, systém však musí v konečnom čase indikovať úspech alebo chybu.

NFP4, rozsah požiadavky. Konkrétne číselné cieľové hodnoty nie sú pre prototyp špecifikované. Požiadavkou je, aby navrhnuté modely a komunikačné rozhrania nemali vo svojej podstate obmedzenia brániace horizontálnemu škálovaniu serverovej vrstvy.

1.3 Analýza fyzického hardvéru

Sekcia popisuje fyzické komponenty IoT koncového uzla. Cieľom nebol priemyselne nasaditeľný zámok, ale funkčný prototyp demonštrujúci end-to-end reťazec systému.

1.3.1 Požiadavky na hardvér

Funkcionalita IoT zariadenia bola z požiadaviek odvodená nasledovne.

- **Konektivita** na serverovú vrstvu. Potrebná je možnosť Wi-Fi pripojenia, nakoľko IoT zariadenie komunikuje s cloudom.
- **Nosič identity čitateľný z mobilného telefónu.** NFC tag s voľne prístupnou pamäťou, ktorú je možné programovať z mikrokontroléra.

- **Vizuálna signalizácia stavu** pomocou LED s možnosťou zmeny farby na indikáciu pripojenia a povoleného, prípadne zamietnutého prístupu.
- **Dostatočný výpočtový výkon a pamäť** pre MQTT klienta nad TLS, ktorý vyžaduje buffery v rádoch kilobajtov a kryptografické operácie.
- **Dostupnosť vývojárskych nástrojov** a podpora prostredia Arduino IDE alebo PlatformIO, existencia knižníc pre MQTT, NFC a LED ovládanie.
- **Jednoduchá montáž bez nutnosti spájkovania** pre rýchle iterácie prototypu.

Tieto kritériá vymedzujú priestor riešení od malých mikrokontrolérov cez výkonnejšie dosky triedy SBC (jednodoskové počítače) až po priemyselne orientované moduly. Nasledujúca pod-sekcia porovnáva relevantných kandidátov.

1.3.2 Porovnanie dostupných platforiem

Pri výbere hardvérového základu boli zvažované štyri kategórie.

Jednočipové mikrokontroléry bez bezdrôtovej konektivity (Arduino Uno, Raspberry Pi Pico) sú lacné a dobre zdokumentované, no vyžadujú externý Wi-Fi modul, ďalšie spájkovanie a komplikujú návrh. **Jednodoskové počítače s Linuxom** (Raspberry Pi Zero 2 W) ponúkajú plnohodnotný operačný systém a vysokoúrovňové vývojárske nástroje [15], no oproti mikrokontrolérom majú zbytočne vyššiu spotrebu [16], dlhší čas spustenia a microSD karta zvyšuje riziko poškodenia pri výpadku napájania. **Priemyselne orientované IoT moduly** (u-blox, Nordic, Particle) sú rádovo drahšie, vyžadujú viazanie na poskytovateľa a ich pridaná hodnota je pre prototyp neopodstatnená. Najvhodnejšou kategóriou sú **mikrokontroléry s integrovanou bezdrôtovou konektivitou**, kde dominujú dve rodiny. **ESP32** od Espressif Systems je mikrokontrolér s 32-bitovým dvojjadrovým procesorom Xtensa LX6 s integrovaným Wi-Fi a Bluetooth, dostupný na desiatkach vývojových dosiek. **Raspberry Pi Pico W** s čipom RP2040 pridáva bezdrôtový modul [17, 18]. Obe spĺňajú požiadavky, rozhodujúcim faktorom je ekosystém. Pre ESP32 existuje oficiálne podporované Espressif Azure IoT SDK aj bohatá Arduino knižničná podpora. ESP32 má navyše hardvérovú akceleráciu TLS 1.2.

1.3.3 Výber vývojovej dosky

Pod rodinou ESP32 boli v užšom výbere tri dosky. Generická **ESP32 DevKit** je najlacnejšia, ale s premenlivou kvalitou bez pohodlných periférnych konektorov. **Adafruit HUZZAH32/Feather ESP32** je o čosi drahšia, prináša integrovaný manažment LiPo batérie a vyššiu kvalitu. **SparkFun IoT RedBoard ESP32** je najdrahšia, ale disponuje navyše USB-C, integrovaným Qwiic konektorom pre I2C periférie, LiPo manažmentom a vstavanou RGB LED [19].

Jedným z rozhodujúcich kritérií bola prítomnosť **Qwiic konektora** priamo na doske. Tento konektor umožňuje pripojenie I2C modulu jedným káblom bez spájkovania, čo pri viacerých iteráciách prototypu výrazne znižuje riziko mechanicky nestabilného zapojenia. Vyššia obstarávacía cena oproti DevKit je vyvážená úsporou času a stabilitou zapojenia.

Doska integruje čip ESP32-D0WDQ6 v module ESP32-WROOM s frekvenciou až 240 MHz, hardvérovou akceleráciou TLS 1.2, USB-C s USB-to-serial prevodníkom a vstavanou LED ovládateľnou cez digitálne rozhranie [19, 20]. Z hľadiska softvéru je plne podporovaná v Arduino IDE aj PlatformIO [21].

1.3.4 NFC modul

Pre nosič identity na strane zámku bol zvolený **SparkFun Qwiic Dynamic NFC/RFID Tag** postavený na čipe **ST25DV64KC** od STMicroelectronics. Pri výbere existujú dve hlavné kategórie. **Statické nálepky** (NTAG213/215/216) majú fixný obsah zapísaný pri výrobe alebo neskôr jednorázovo a sú extrémne lacné, ale nedovoľujú mikrokontroléru obsah prepisovať. **Dynamické duálne tagy** sprístupňujú pamäť cez RF rozhranie aj cez I2C zbernicu z hostiteľského mikrokontroléra, čo umožňuje dynamický prepis obsahu zo strany firmvéru.

Kľúčové parametre zvoleného modulu sú dvojportová architektúra (RF + I2C), 64kbit EEPROM, zhoda so štandardom ISO/IEC 15693 a NFC Fórum Type 5, schopnosť čítania pri úplnom výpadku napájania a integrovaná PCB anténa s dosahom niekoľko centimetrov [22]. V navrhovanom systéme sa dynamická vlastnosť využíva pri inicializácii zariadenia, kedy mikrokontrolér zapíše do tagu identifikátor zariadenia vo formáte NDEF Text record. Voľba SparkFun varianty s Qwiic konektorom namiesto alternatív s rovnakým čipom (napríklad X-NUCLEO-NFC04A1) bola motivovaná jednoduchým prepojením s ESP32 zariadením a dostupnosťou výrobcom poskytovanej knižnice.

1.3.5 Hardvérová zostava ako celok

Výsledná zostava pozostáva z dosky SparkFun IoT RedBoard ESP32 a SparkFun Qwiic Dynamic NFC Tag prepojených jedným Qwiic káblom. Zapojenie je fyzicky nezameniteľné a nevyžaduje spájkovanie. Napájanie je cez USB-C port dosky. Elektromechanické otváranie zámku je simulované LED signalizáciou.

1.4 Analýza softvérového stacku

Systém pozostáva z piatich softvérových komponentov (mobilná aplikácia, serverová vrstva s databázou a proxy komponentom, cloudová IoT vrstva a firmvér mikrokontroléra). Sekcia dokumentuje rozhodnutia o voľbe konkrétnych technológií pre každú vrstvu.

1.4.1 Mobilná aplikácia

Požiadavka multiplatformnosti určuje, že aplikácia musí fungovať na iOS aj Android. Existujú tri hlavné prístupy k riešeniu tejto požiadavky.

Natívny vývoj v dvoch samostatných kódových základoch

Swift pre iOS a Kotlin pre Android poskytujú najlepšiu integráciu s operačným systémom, ale celú biznis logiku, dátové modely a UI je nutné implementovať a udržiavať dvakrát. Pre prototyp je tento prístup neopodstatnene nákladný.

React Native

React Native od spoločnosti **Meta** je cross-platformový framework v JavaScripte/TypeScripte, vykresľujúci natívne komponenty operačného systému. Je vhodný pre aplikácie zložené prevažne zo štandardných UI prvkov a pre tímy s JavaScript zázemím.

Flutter

Flutter je open-source framework od **Google** postavený na jazyku Dart, ktorý kompiluje aplikácie AoT do natívneho strojového kódu cieľovej platformy a UI vykresľuje cez vlastnú vykresľovaciu vrstvu Impeller. Google uvádza typický podiel zdieľaného kódu medzi platformami 90 % [23]. Voľba padla na Flutter z nasledujúcich dôvodov:

- Vysoké zdieľanie biznis logiky a UI medzi iOS a Android pri podobnom rozhraní pre obe platformy.
- Vyspelá podpora natívneho kódu cez *platform channels* [24], kritická pre integráciu NFC funkcionality, ktorá vyžaduje volania do natívu.
- Dobre podporovaný Dart klient pre Supabase.
- Aktívny ekosystém balíčkov pre statový manažment, navigáciu, lokalizáciu a bezpečné úložisko.

1.4.2 Serverová vrstva

Pre serverovú vrstvu boli zvažované tri hlavné alternatívy, vývoj vlastného backendu, použitie platformy Firebase, alebo platformy Supabase.

Vlastný backend

Poskytuje maximálnu kontrolu nad architektúrou a neexistuje pri ňom viazanosť na dodávateľa. Cenou je však významné navýšenie implementačnej práce. Autentifikácia, správa používateľov, generovanie JWT, RESTful API vrstva, systém migrácií schémy, nasadenie, monitorovanie a škálovanie musia byť explicitne riešené. Pre prototyp mobilnej aplikácie je tento prístup neefektívny.

Firebase

Ide o dlhoročne zavedenú BaaS platformu od **Google**, ktorá poskytuje autentifikáciu, databázu na báze NoSQL, serverové funkcie a ďalšie služby. Jej hlavné obmedzenia vzhľadom na požiadavky tohto systému sú dve. Dátový model navrhovaného systému je striktne relačný a vyhodnocovanie prístupových práv pri odomykaní si vyžaduje agregované dotazy naprieč viacerými kolekciami. Oba vzory dotazov, ktoré sa v NoSQL databázach, ktoré Firebase ponúka, implementujú nepohodlne a s výkonovými kompromismi.

Supabase

Supabase je open-source platforma pozicionovaná ako alternatíva k Firebase, postavená nad PostgreSQL [25]. Poskytuje plnohodnotnú SQL databázu, autentifikačný systém s JWT, automaticky generované REST/GraphQL API, Edge Functions v Deno runtime a natívnu podporu RLS. Pre navrhovaný systém boli rozhodujúce dva aspekty. PostgreSQL umožňujúce prirodzene modelovať relačnú hierarchickú štruktúru a RLS umožňuje deklaratívne politiky prístupu na úrovni databázy, čo presúva autorizačnú logiku z aplikačnej vrstvy. Detaily použitia sú v druhej kapitole.

1.4.3 Cloudová IoT vrstva

Pre komunikáciu medzi serverovou vrstvou a IoT zariadením boli zvažované tri kategórie riešení. Cloudová IoT platforma od veľkých poskytovateľov, vlastný nasadený MQTT broker alebo priame HTTP spojenie medzi serverom a zariadením.

Priame HTTP spojenie

Toto riešenie je síce jednoduché, ale nedá sa ním rozumne riešiť scenár, keď zariadenie je za NAT a server potrebuje iniciovať komunikáciu smerom k nemu. Zariadenie by muselo mať verejnú IP alebo používať dlhotrvajúci HTTP polling, čo je neefektívne a vyžaduje explicitnú správu spojenia na strane zariadenia.

Vlastný MQTT broker

Broker rieši smer z cloudu na zariadenie elegantne, kedy zariadenie je odberateľom a broker mu doručí správu, ale vyžaduje správu samotnej brokerovej infraštruktúry, jej nasadenie, zabezpečenie, monitorovanie a škálovanie. Podobne ako pre backend je toto zbytočná réžia navyše.

Azure IoT Hub

IoT Hub je plne spravovaná služba **Microsoftu**, ktorá pokrýva presne tento scenár [26]. Ponúka nasledovné relevantné vlastnosti.

- **Obojsmerná komunikácia** medzi cloudom a zariadením, s podporou protokolov MQTT, AMQP a HTTP [27].
- **Direct Methods** sú synchrónne vzdialené volania z cloudu na konkrétne zariadenie s nastaviteľným časovým limitom [28]. Pre príkaz odomknutia ide o ideálny mechanizmus v rámci jedinej synchrónnej operácie.
- **Device Twin** je cloudový záznam o každom zariadení s nastaviteľnými vlastnosťami, umožňujúci správu konfigurácie zariadenia a sledovanie jeho stavu.
- **Autentifikácia zariadenia** cez SAS tokeny alebo X.509 certifikáty [27].
- **Šifrovanie komunikácie** cez TLS 1.2 je povinné, port 8883 pre MQTT [27].

- **Štandardizované MQTT volania.** Zariadenie prijíma volania a odpovedá na predom definovanom odkaze [28], čo umožňuje implementovať klienta aj bez použitia oficiálnych Azure IoT SDK, priamo cez ľubovoľnú MQTT knižnicu.

Alternatívny **AWS IoT Core** od Amazonu ponúka funkčne porovnateľné možnosti a v obchodnej ponuke je zrovnateľný. Voľba Azure IoT Hub bola motivovaná dostupnosťou bezplatného študentského kreditu Azure v rámci akademickej licencie Microsoft Azure for Students a praktickou skúsenosťou s Azure prostredím.

1.4.4 Firmvér IoT zariadenia

Pre firmvér ESP32 boli v praxi dostupné tri programovacie prostredia. **ESP-IDF**, natívne C/C++ SDK od Espressif Systems, **Arduino framework**, C++ nadstavba s rozsiahlou komunitnou knižničnou podporou a **MicroPython**, interpretovaný Python bežiaci priamo na čipe zariadenia.

MicroPython bol zamietnutý pre jeho interpretovanú povahu. Réžia interpretovania Pythonu je na embedded zariadení citeľná pri výkonnostne citlivých úlohách ako MQTT nad TLS a pri paralelnom spracovaní NFC. ESP-IDF je najvýkonnejší a dáva najvyššiu kontrolu, jeho použitie však vyžaduje hlbšie porozumenie Real-Time Operating System konceptov a významnejší čas na zoznámenie.

Arduino framework bol zvolený ako rozumný kompromis, je postavený nad ESP-IDF, takže neobetuje výkon, ponúka rozsiahlu a dobre udržiavanú knižničnú podporu vrátane PubSubClient pre MQTT a Adafruit NeoPixel pre ovládanie LED a je priamo kompatibilný so SparkFun vývojovými doskami, vrátane zvolenej SparkFun IoT RedBoard. Pre NFC tag ST25DV64KC je dostupná oficiálna SparkFun Arduino knižnica, čo bolo rozhodujúce pri praktickej implementácii.

1.4.5 Zhrnutie softvérového stacku

Celkový softvérový stack systému teda pozostáva z nasledujúcich technológií

1. **Mobilná aplikácia.** Flutter s natívnymi knižnicami v Swift a Kotlin.
2. **Serverová vrstva.** Supabase s PostgreSQL databázou, autorizáciou a Edge Functions.
3. **Sieťová medzivrstva.** Cloudflare Worker.
4. **IoT cloud.** Azure IoT Hub s podporou MQTT, direct methods a device twins.
5. **Firmvér zariadenia.** Arduino framework.

1.5 Koncepty hardvérového stacku a NFC komunikácie

Pri NFC komunikácii medzi telefónom a zámkom existuje niekoľko architektonicky odlišných konceptov rozdelenia rolí. Sekcia popisuje tri modely, ktoré

boli počas vývoja zvažované. Výber modelu determinuje celý zvyšok systémového návrhu, od hardvéru po mobilnú aplikáciu, takže dokumentácia tohto procesu je neoddeliteľnou súčasťou analýzy.

Pred popisom prototypov je potrebné zadať dva relevantné NFC režimy. **Card emulation mode**, v ktorom zariadenie emuluje bezkontaktnú kartu a vystupuje pasívne voči čítačke a **reader/writer mode**, v ktorom zariadenie aktívne číta obsah cieľového NFC tagu. Tretí režim NFC, **peer-to-peer**, nie je pre danú úlohu relevantný a na moderných platformách obmedzený alebo odstránený.

1.5.1 Mobilný telefón ako emulovaný NFC tag

Prvý koncept vychádzal z intuitívne najprirodzenejšej analógie s fyzickým kľúčom, kde telefón vystupuje ako pasívny nosič identity a zámok obsahuje NFC čítačku. Pri priložení telefónu k zámku čítačka načíta kľúč uložený v aplikácii. Tento model je na platforme Android implementovaný cez mechanizmus HCE, ktorý Google zaviedol v Android 4.4 [29]. HCE umožňuje ľubovoľnej aplikácii emulovať NFC kartu na softvérovej úrovni bez nutnosti hardvérového Secure Element, pričom NFC kontrolér smeruje komunikáciu priamo do aplikácie, ktorá odpovedá cez **Application Protocol Data Unit** správy podľa ISO/IEC 7816-4. Aplikácia môže byť identifikovaná prostredníctvom zaregistrovaného identifikátora aplikácie.

Z hľadiska požiadavky na multiplatformnosť aplikácie sa tento prístup ukázal ako nevhodný. Na platforme iOS bolo HCE dlhodobo vývojárom tretích strán nedostupné. Apple sprístupnil NFC API tretím stranám prvýkrát na konferencii WWDC 2017 v rámci frameworku CoreNFC [30], pôvodne iba na čítanie NDEF tagov. Kartová emulácia bola naďalej rezervovaná iba pre Apple Pay prostredníctvom Secure Element, bez možnosti otvorenia tretím stranám.

Situácia sa zmenila až v roku 2024 v kontexte regulačného tlaku Európskej únie. Po niekoľkoročnom antimonopolnom vyšetrovaní Apple v januári 2024 ponúkol Európskej komisii záväzky na otvorenie prístupu k NFC pre tretie strany cez HCE režim, a v júli 2024 tieto záväzky komisia prijala ako právne záväzné po dobu desiatich rokov [31]. V iOS 17.4 následne Apple sprístupnil HCE API pre vývojárov tretích strán, ale s prísnymi obmedzeniami.

1. HCE je dostupné iba v Európskom hospodárskom priestore.
2. Vyžaduje explicitnú HCE Entitlement udelenú Apple na základe žiadosti, ktorú vývojár podá cez Apple Developer program.
3. Udelenie povolenia podlieha regulačným a bezpečnostným podmienkam, splnenie štandardov, ako sú rôzne compatibility pri platbách a otázke GDPR, existencia procesov na spracovanie bezpečnostných incidentov a aj regulačné povolenie podľa typu aplikácie [32].

Uvedené podmienky sú primárne zamerané na platobné aplikácie a predstavujú pre nezávislý študentský projekt v oblasti kontroly prístupu neprekonateľnú bariéru. Aj keby sa žiadosť o HCE povolenie teoreticky podarilo získať, samotný fakt, že funkcionality by bola dostupná len vo vymedzenom priestore by porušovala požiadavku na univerzálne funkčný prototyp. Z tohoto dôvodu bol prvý prototyp zamietnutý ako neimplementovateľný.

1.5.2 ESP32 v kombinácii s NFC modulom PN532

Druhý koncept priniesol zásadný architektonický posun a to obrátenie rolí. Na rozdiel od prvého prototypu už nešlo o emuláciu kľúča v telefóne, ale o emuláciu NFC tagu na strane zámku, ktorý by telefón v úlohe čítačky jednoducho prečítal. Touto inverziou sa eliminuje závislosť od HCE API operačného systému telefónu, jednoduché čítanie pasívneho NDEF tagu je na strane iOS aj Android štandardná, dlhodobo dostupná operácia bez špeciálnych povolení alebo regulačných obmedzení [33, 29]. Architektonicky je teda tento smer správnym riešením problému prvého konceptu.

Konkrétnou hardvérovou voľbou pre realizáciu tohto konceptu bol **PN532**, jeden z najrozšírenejších NFC modulov v amatérskom a študentskom ekosystéme. Voľba bola motivovaná dvomi praktickými dôvodmi. Ide o široko dostupný produkt za nízku cenu a je bežne používaný v podobných projektoch. Z technického hľadiska PN532 podľa svojej špecifikácie podporuje všetky tri NFC režimy, reader/writer, peer-to-peer a aj emulácia karty [34] a preto sa javil ako univerzálne riešenie pokrývajúce potreby projektu.

ESP32 v tomto variante riadi PN532 cez SPI rozhranie, prepne ho do režimu emulácie karty a inicializuje ho ako NFC Fórum Type 4 tag s NDEF obsahom obsahujúcim identifikátor zariadenia. Telefón v úlohe čítačky následne pri priložení tag prečíta a získané UUID odošle do serverovej vrstvy na autorizáciu.

V praktickom testovaní sa však ukázalo, že režim emulácie trpí systematickou nespoľahlivosťou vyplývajúcou z viacerých príčin.

- **Komunikácia ESP32 a PN532.** Pri opakovaných pokusoch nebol PN532 občas detegovaný napriek korektnému elektronickému zapojeniu. Taktiež boli vyskúšané aj iné typy rozhrania komunikácie, ako napríklad I2C, kde sa vo výsledku problém neodstránil.
- **Nespoľahlivá detekcia emulovaného tagu telefónmi.** Aj v prípadoch, keď bol PN532 úspešne inicializovaný a vstúpil do emulačného režimu, samotné prečítanie tagu telefónom prebiehalo iba v mizivom percente pokusov. Príčina problému je dobre zdokumentovaná aj v širšej NFC literatúre. Coskun a kol. [35] v prehľadovej práci o NFC technológii poukazujú na to, že kompatibilita medzi rôznymi NFC frontendmi v telefónoch a softvérovou emuláciou tagov je v praxi obmedzená rozdielmi v polling sekvenciách a tolerancii časových odchýliek jednotlivých čítačiek. V praxi sa pozoruje, že tá istá emulovaná konfigurácia funguje na jednom telefóne spoľahlivo z polovice pokusov a na inom takmer vôbec, prípadne sa tag identifikuje s nesprávnym typom. Rovnaký jav opakovane reportujú aj príspevky vo vývojárskych komunitách [36, 37]. Modul, navrhnutý primárne ako NFC čítačka, na túto šírku frontendovej variability nebol optimalizovaný.
- **NXP odporúčanie pre emuláciu.** V oficiálnej technickej komunite NXP sa pri otázkach o stabilite režimu emulácie karty PN532 objavuje odporúčanie zvoliť modernejšie kontroléry, ktoré sú špecificky stavané na robustnú emuláciu tagov [38]. PN532 je teda hardvérom všeobecného určenia, vhodným predovšetkým na experimentálne čítanie tagov, nie na produkčne stabilnú emuláciu.

V praktickom testovaní sa kombinácia týchto problémov prejavila tak, že pomer úspešných čítaní k celkovému počtu pokusov, bol natoľko nízky, že systém by bol pre koncového používateľa skôr frustrujúci ako použiteľný. Hlavná architektonická myšlienka, obrátenie rolí, sa však ukázala ako správna. Problém bol výhradne v zvolenej hardvérovej realizácii. Pri hľadaní stabilnejšej alternatívy bol identifikovaný komponent, ktorý je explicitne navrhnutý pre tento scenár a to dynamický NFC tag, ktorého obsah je možné meniť zo stany hostiteľského mikrokontroléra cez I2C, ale samotné rádiové rozhranie je realizované ako pasívny tag bez emulácie. Tento smer viedol k tretiemu a finálnemu prototypu.

1.5.3 Využitie dynamického NFC tagu

Tretí a finálny koncept zachoval architektonický princíp z druhého neúspešného konceptu, zámok obsahuje NFC tag a telefón vystupuje ako čítačka. Koncept zmenil hardvérovú realizáciu zo strany zámku a aktívne emulovaného tagu na pasívny dynamický NFC tag. Konkrétne použitým komponentom je SparkFun Qwiic Dynamic NFC/RFID Tag postavený na čipe ST25DV64KC, ktorý výrobca priamo deklaruje ako určený na tento typ použitia. Hostiteľský mikrokontrolér zapíše obsah do EEPROM cez I2C a tag následne tento obsah pasívne sprístupňuje ľubovoľnej NFC čítačke v dosahu rádiového poľa [39].

Voľba tohto konkrétneho komponentu priniesla okrem hlavného prínosu (technickej stability) aj nové praktické výhody.

Komponentová jednota cez Qwiic ekosystém. Hlavný mikrokontrolér aj NFC tag pochádzajú od rovnakého výrobcu a používajú jednotnú **Qwiic** zbernicu, štandard SparkFun postavený na I2C [40]. Celé fyzické zariadenie tak vznikne spojením dvoch dosiek jediným Qwiic káblom, bez potreby spájkovania, prepojení vodičov či riešenia úrovnových konverzií napätí. Toto výrazne znižuje hardvérovú zložitosť a riziko zlyhania na úrovni elektrického prepojenia, ktoré bolo kritickou slabinou druhého konceptu.

Spolahlivosť čítania. Pasívny NDEF tag predstavuje najjednoduchšiu a najstabilnejšiu formu NFC interakcie. Telefón pri priblížení vyrobí elektromagnetické pole, ktoré tag napája, a jednosmerne z neho prečíta preddefinovaný obsah. Nedochádza k aktívnej výmene príkazov medzi dvoma riadenými zariadeniami. V testovaní bola úspešnosť čítania pri správnom priložení prakticky bezchybná.

Bezpečnostný posun autorizácie do cloudu. Paradoxne, práve zdanlivý nedostatok tohto modelu, nutnosť online pripojenia pri každom odomknutí, je z hľadiska bezpečnosti výhodou. Pri architektúre prvého konceptu by bolo potrebné dôverovať kryptografickému protokolu medzi telefónom a zámkom, spravovať relačné kľúče na oboch stranách a riešiť ich obnovu. V navrhovanom modeli je identifikátor zariadenia verejný, ale autorizácia prebieha mimo fyzickej NFC interakcie. Kompromitácia jedného telefónu alebo jedného tagu nespôsobí kolaps systému ako celku. Revokácia prístupu je otázka zmeny záznamu v databáze s okamžitým účinkom.

Technickou realizáciou tohto konceptu je teda dvoj-komponentová zostava SparkFun IoT RedBoard ESP32 ako hostiteľský mikrokontrolér a SparkFun Dynamic NFC/RFID tag ako pasívny tag, prepojené jedným Qwiic káblom. Na strane mobilného telefónu je na iOS využité CoreNFC v API cez natívny plugin v jazyku Swift a na Android NfcAdapter API cez natívny plugin v jazyku Kotlin, pričom

oba sú z Dart vrstvy Flutter aplikácie prístupné cez unifikovaný platform channel. Detaily implementácie sú rozpísané v tretej kapitole.

1.5.4 Zhrnutie iteratívneho vývoja

Iteratívny proces tak postupne vyradil dve architektúry a finálne riešenie prinieslo to, čo predošlé nedokázali. Cloudovú autorizáciu, multiplatformnú kompatibilitu bez regulačných obmedzení a výrazne zjednodušenú hardvérovú zostavu.

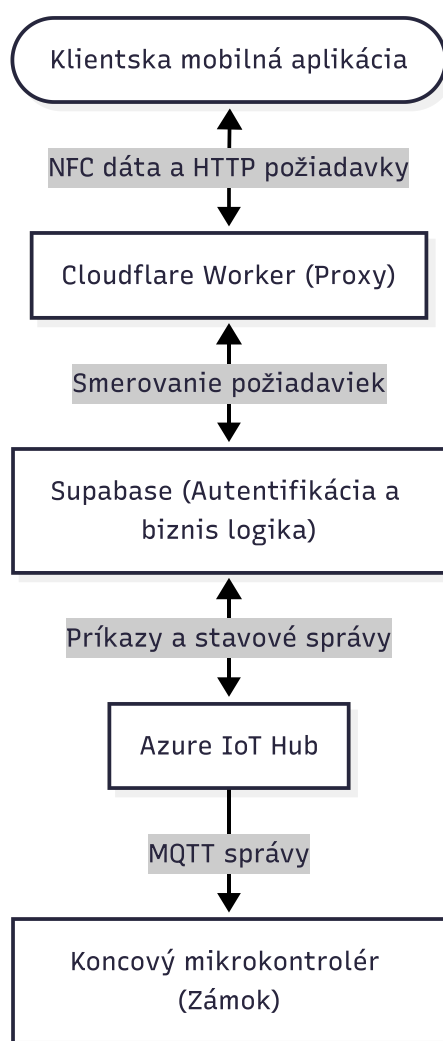
2 Návrh systému a architektúry

Kapitola popisuje detailný návrh systému, jeho architektúru a topológiu komponentov, dátový model s dôrazom na multitenanciu a hierarchické organizačné štruktúry, priebeh procesu odomykania a bezpečnostný návrh.

2.1 Architektúra systému a topológia

Navrhovaný systém je postavený na distribuovanom modeli. Žiadny z jeho komponentov neposkytuje samostatnú kompletnú funkcionálnu a samotná funkčnosť celku je závislá od kooperácie všetkých častí. Rozhodovanie o autorizácii používateľa je centralizované v cloudovej vrstve, čím sa zodpovednosť systému za bezpečnosť presúva z fyzického zariadenia aj z mobilnej aplikácie.

2.1.1 Architektúra komponentov



Obr. 2.1 Abstraktný pohľad na architektúru systému

Systém pozostáva z piatich logicky oddelených komponentov, kde každý plní vopred zadefinovanú úlohu v komunikačnom reťazci od používateľa po fyzický zámok (viď obrázok 2.1).

Mobilná aplikácia je klientskou vrstvou systému. Beží na používateľskom zariadení, či už ide o iOS alebo Android, poskytuje používateľské rozhranie pre autentifikáciu, prehľad pridelených kľúčov a administratívne úkony. Zároveň je jediným vstupným bodom pre interakciu používateľa so zámkom. Pri odomýkaní aplikácia plní najskôr rolu čítania NFC tagu fyzického zariadenia, ktorá je zabezpečená prostredníctvom natívneho pluginu využívajúceho CoreNFC alebo NfcAdapter. Po úspešnom prečítaní vytvára autorizačnú požiadavku smerom do serverovej vrstvy.

Cloudflare Worker je tenká serverless proxy vrstva bežiaca v sieti Cloudflare. Jej jedinou funkciou je zabezpečiť sieťovú kompatibilitu medzi klientom a Supabase. Z pohľadu aplikácie táto komponenta nič nepridáva k biznis logike, pôsobí transparentne a jej nasadenie alebo vynechanie je riadené konfiguračnou premennou v aplikácii.

Supabase predstavuje primárnu backendovú vrstvu systému a zodpovedá za autentifikáciu používateľov, perzistentnosť dát, implementáciu prístupových politík na úrovni databázy a hostovanie serverless *Edge Functions*, ktoré zabezpečujú komplexnejšie operácie vyžadujúce orchestráciu viacerých podsystémov. V kontexte tejto práce je najdôležitejšia funkcia **unlock-device**, ktorá vyhodnotí autorizáciu cez databázové RPC volanie a výsledok smeruje cez Azure IoT Hub a taktiež vracia informáciu späť do používateľského zariadenia.

Azure IoT Hub je cloudová IoT platforma zabezpečujúca obojsmerný komunikačný kanál medzi Supabase a fyzickým zariadením. V tomto reťazci umožňuje serveru iniciovať volanie smerom k zariadeniu, aj keď je za NAT a nemá verejnú IP adresu, prostredníctvom *direct methods*. Samotné zariadenie je pripojené trvalým MQTT spojením na porte 8883 cez TLS [27]. Bez tejto sprostredkujúcej vrstvy by bola komunikácia *cloud-to-device* prakticky nerealizovateľná, bez rôznych zásahov do siete klienta.

IoT zariadenie je koncový uzol systému a je fyzicky umiestnený v konkrétnej budove. Zariadenie má viacero úloh.

1. Pri spustení zariadenia sa pripojí k Wi-Fi a následne k Azure IoT Hubu.
2. Udržiava aktívne MQTT spojenie.
3. Počúva prichádzajúce *direct method* volania zo strany serveru.
4. Pomocou LED vhodne reaguje na prijaté volania a signalizuje výsledok.

2.1.2 IPv4 a IPv6 kompatibilita

Nefunkčná požiadavka NFP7 vyžaduje funkčnosť systému aj v sieťach bez podpory IPv6. Mnohé verejné a inštitucionálne Wi-Fi siete dodnes IPv6 nepodporujú, no Supabase vo svojom štandardnom nasadení akceptuje priame pripojenie výhradne cez IPv6 adresu [41]. Pre IPv4 siete dokumentácia odporúča použiť Supabase pooler alebo platený IPv4 Add-On [42]. Mobilná aplikácia spoliehajúca sa na priame IPv6 pripojenie by tak v týchto prostrediach zlyhala už pri pokuse o autentifikáciu.

Riešením je zavedenie medzivrstvy vo forme **Cloudflare Workera**, serverless platformy bežiaccej na globálnej edge sieti Cloudflare s podporou IPv4 aj IPv6. Worker prijíma HTTPS požiadavky od aplikácie cez IPv4 a presmeruje ich na Supabase cez IPv6, bez prístupu k obsahu požiadaviek. Implementačné detaily a meranie pridanej latencie sú v tretej a štvrtej kapitole.

2.1.3 Centralizovaná autorizácia

Hlavným architektonickým princípom celého systému je centralizovanie autorizácie na serverovú vrstvu. Mobilná aplikácia ani fyzický zámok oprávnenia nevyhodnocujú. Klient len odošle UUID cieľového zariadenia, server overí oprávnenie cez prístupový kľúč používateľa a doručí zariadeniu výsledný príkaz. Tento princíp prináša tri kľúčové výhody:

- **Manipulácia s klientom alebo zámkom neumožní zmenu autorizačnej politiky**, keďže žiadna z týchto vrstiev autorizačnú logiku neobsahuje a ani cudzie kľúče.
- **Revokácia je okamžitá** a každá zmena prístupu sa prejaví pri najbližšej požiadavke bez potreby synchronizácie do koncových zariadení.
- **Audit a centrálna kontrola**, všetky autorizačné rozhodnutia prebiehajú na jednom mieste.

Cenou tohto modelu je závislosť na sieťovom pripojení. Bez spojenia so serverovou vrstvou zariadenie nedokáže reagovať na žiadosti o odomknutie.

2.2 Návrh databázového modelu

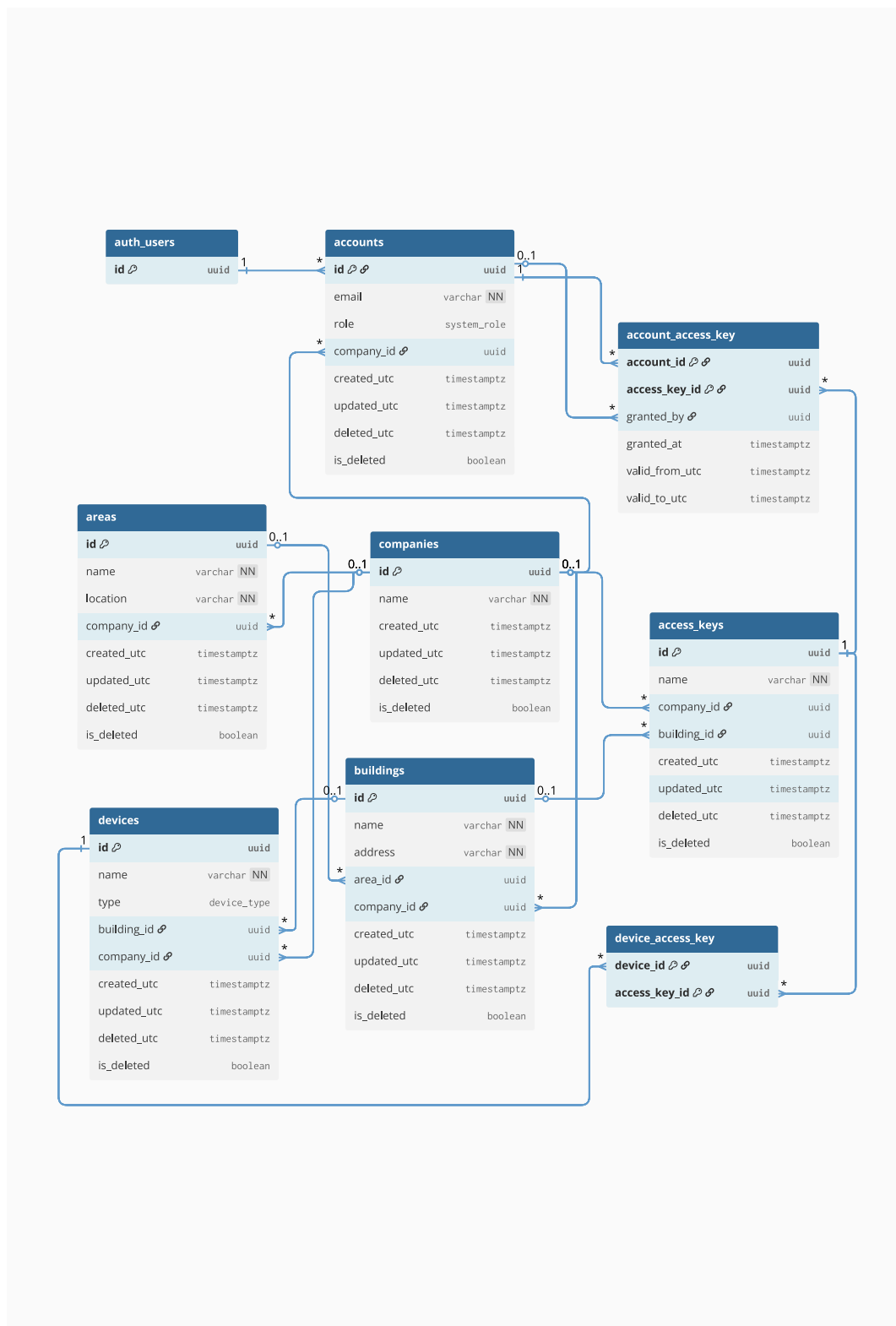
Dátový model systému je navrhnutý ako hierarchická relačná štruktúra s oddelenou vrstvou oprávnení postavenou na modeli RBAC (viď ER diagram 2.2). Tento model pozostáva z ôsmich používateľských tabuliek rozdelených do troch konceptuálnych skupín so spojovými tabuľkami s M:N väzbami.

- **Organizačná štruktúra.** Tabuľky `companies`, `areas`, `buildings`, `devices`.
- **Používatelia.** Tabuľka `accounts`.
- **Oprávnenia.** Tabuľka `access_keys`.
- **Väzby medzi tabuľkami.** `account_access_key`, `device_access_key`.

2.2.1 Princípy návrhu

Model sa riadi štyrmi hlavnými princípmi, viditeľnými v celej štruktúre.

Multi-tenancy na úrovni spoločnosti. Každá entita obsahuje cudzí kľúč `company_id`, odkazujúci na tabuľku `companies`. Keďže pri každej požiadavke je k používateľovi pripojený jeho `company_id`, RLS dokáže jednoducho filtrovať všetky záznamy. Tým je zabezpečené, že prihlásený používateľ vidí výhradne dáta svojej spoločnosti. V hlbších úrovniach hierarchie, ako napríklad v tabuľke `devices`,



Obr. 2.2 Databázová schéma

ktorá obsahuje aj `building_id`, sú prítomné aj iné cudzie kľúče, ktoré odľahčujú výpočty pri frekventovaných operáciách.

Hierarchická stromová štruktúra. Entity v tabuľkách `companies`, `areas`, `buildings` a `devices` tvoria stromovú štruktúru, kde každá nižšia úroveň patrí práve jednej nad ňou. Každé zariadenie odkazuje na práve jednu budovu, ktorá odkazuje na práve jednu oblasť a tá odkazuje na práve jednu spoločnosť. Hierarchia umožňuje intuitívne modelovať fyzický svet, kde jedna spoločnosť prevádzkuje viaceré oblasti, do ktorých spadá niekoľko budov, v rámci ktorej je umiestnených niekoľko zariadení.

Soft delete. Každá hlavná tabuľka obsahuje dvojicu stĺpcov `is_deleted` a `deleted_utc`. Vymazanie záznamu tým pádom nevedie k trvalému a nenávratnému odstráneniu entity, ale iba zmení flag v rámci záznamu na `true` a zaznamená čas vykonania akcie. Princíp slúži na vrátenie omylom zmazaných záznamov a zachovanie historických záznamov.

Timestampy. Každá tabuľka obsahuje stĺpec `created_utc` a `updated_utc`. Tieto záznamy sú spravované cez databázové triggery a sú dôležité pre prípadnú analýzu udalostí a pre priebežnú kontrolu.

2.2.2 Organizačná štruktúra

Štyri tabuľky reprezentujú stromovú hierarchiu `companies` → `areas` → `buildings` → `devices`. Každá obsahuje primárny kľúč typu `UUID`, generovaný náhodne, čím sa eliminuje predikovateľnosť identifikátorov. Okrem informačných stĺpcov (názov, lokácia, typ zariadenia) obsahujú aj cudzí kľúč na priamo nadradenú entitu a duplicitný odkaz na `company_id` pre rýchle filtrovanie cez RLS politiky. Štandardné stĺpce `created_utc`, `updated_utc` a `is_deleted` sú prítomné vo všetkých tabuľkách.

2.2.3 Entity používateľov

Accounts. Tabuľka `accounts` reprezentuje používateľský účet v aplikačnej vrstve. Jej primárny kľúč `id` je totožný s identifikátorom v tabuľke `auth.users`, ktorá je spravovaná Supabase Auth službou. Toto prepojenie umožňuje aplikačnej vrstve operovať nad tým istým identifikátorom s akým pracuje autentifikačná vrstva Supabase bez manuálnej synchronizácie. Okrem primárneho identifikátora obsahuje tabuľka e-mail, ktorý je duplikovaný z autentifikačnej tabuľky Supabase, cudzí kľúč `company_id` odkazujúci na spoločnosť a stĺpec `role`. Tento stĺpec určuje systémovú rolu používateľa, teda aké administratívne privilégia používateľ v aplikácii má.

2.2.4 Entita prístupových kľúčov

Access_keys reprezentuje digitálny kľúč ako pomenovanú množinu zariadení. Obsahuje primárny kľúč, názov, cudzí kľúč na spoločnosť a štandardné stĺpce.

Dôvodom voľby tejto abstrakcie pred priamym priradením zariadení používateľom je efektivita opakovaných priradení. V budove existuje typicky obmedzený počet prirodzených skupín zariadení, ktoré sa opakovane pridelujú meniacim sa používateľom. Bez tejto abstrakcie by správca pri každom novom hostovi musel

manuálne vyberať desiatky jednotlivých zariadení. S abstrakciou je definícia skupiny jednorázovým úkonom pri zriadení budovy a pridelenie novému používateľovi vyžaduje výber jednej položky.

2.2.5 Spojové tabuľky

Pri realizácii M:N vzťahov obsahuje navrhnutá schéma tri spojové tabuľky.

account_access_key. Realizuje vzťah *používateľ má pridelený prístupový kľúč*. Mapuje používateľa `account_id` k prístupovému kľúču `access_key_id`. Navyše má časové atribúty ohraničujúce začiatok `valid_from_utc` a koniec `valid_to_utc` platnosti pridelenia. Pre uľahčenie spätnej analýzy, v prípade potreby, sú prítomné aj stĺpce `granted_by` a kedy `granted_at` vykonal priradenie. Posledný stĺpec `revoked_at` informuje o čase odňatia prístupu.

device_access_key. Realizuje vzťah *prístupový kľúč má prístup k zariadeniu*. Mapuje kľúč `device_id` k `access_key_id`. Tento vzťah je čisto štrukturálny, bez ďalších atribútov.

2.2.6 Enumeračné typy

Vďaka tomu, že PostgreSQL podporuje vlastné enumeračné typy, je možné si zadať nové pre typovú bezpečnosť v databáze. V návrhu sú konkrétne využité nasledujúce typy.

- **system_role** určuje hodnoty `owner`, `manager`, `user`, používané v stĺpci `accounts.role`.
- **device_type** definuje typ fyzického zariadenia, či už ide o dvre, turniket, bránu a podobne.

2.2.7 Prepojenie s autentifikačnou vrstvou

Supabase Auth spravuje používateľov v internej schéme `auth.users`. Pri registrácii sa najprv vytvorí záznam v `auth.users` s e-mailom, zahashovaným heslom, následne v aplikačnej schéme `public.accounts` so zhodným identifikátorom. Cudzí kľúč `accounts.id` → `auth.users.id` zaručuje, že každý aplikačný účet má svoj zrkadlový záznam v Supabase autentifikačnej tabuľke. Návrh tým získava flexibilitu rozšírenia účtu o ľubovoľné aplikačné stĺpce mimo prednastavenej Supabase schémy.

2.3 Komunikačný tok odomykacieho procesu

Proces odomknutia je centrálna interakcia, okolo ktorej je celý systém navrhnutý. Každý ďalší komponent, či už je to hierarchická správa organizácií, priradenie rolí, kľúče s časovo ohraničeným oknom, alebo bezpečnostný model, je len súčiastka komplexného rozhodovacieho procesu, či konkrétny používateľ v danom čase môže odomknúť prilahlé zariadenie. Táto sekcia detailne opisuje, ako tento rozhodovací proces prebieha *pod povrchom*, od momentu kedy používateľ iniciuje čítanie NFC zariadenia, až po vizuálnu signalizáciu zámku. Tento proces si

rozoberieme detailne z troch pohľadov, aplikačnej, serverovej a z pohľadu fyzického zámku.

2.3.1 Celkový tok

Proces samotný prebieha v šiestich synchrónne reťazených fázach.

1. **Načítanie NFC tagu** na strane mobilnej aplikácie.
2. **Autorizačná požiadavka** z aplikácie do serverovej vrstvy.
3. **Validácia JWT** a identifikácia používateľa na strane Supabase.
4. **Vyhodnotenie prístupových práv** v databáze cez RPC funkciu.
5. **Iniciácia direct method** voči Azure IoT Hub.
6. **Fyzická reakcia zariadenia** pomocou LED na ESP32.

Z pohľadu používateľa je celý proces jednou zdanlivo okamžitou operáciou, technicky však prebiehajú jednotlivé fázy sekvenčne cez tri rôzne komunikačné protokoly a tri infraštruktúry. Konkrétne časové rozpočty po fázach sú vyhodnotené v štvrtej kapitole.

2.3.2 Pohľad mobilnej aplikácie

Mobilná aplikácia obsluhuje prvú a druhú fázu komunikačného toku (čítanie NFC tagu a odoslanie autorizačnej požiadavky) a po vrátení odpovede zo serverovej vrstvy aj záverečné zobrazenie výsledku (viď diagram 2.3).

Prvá fáza, čítanie NFC tagu. Používateľ ťukne na interaktívne tlačidlo na hlavnej obrazovke, ktoré cez platform channel zavolá natívnu metódu `readDeviceId`. Na iOS sa otvorí systémový NFC modál, na Androide sa aktivuje foreground dispatch. Po úspešnom prečítaní NDEF tagu natívna vrstva vráti do Dart kódu UUID zariadenia. Pri zlyhaní je vrátený chybový kód.

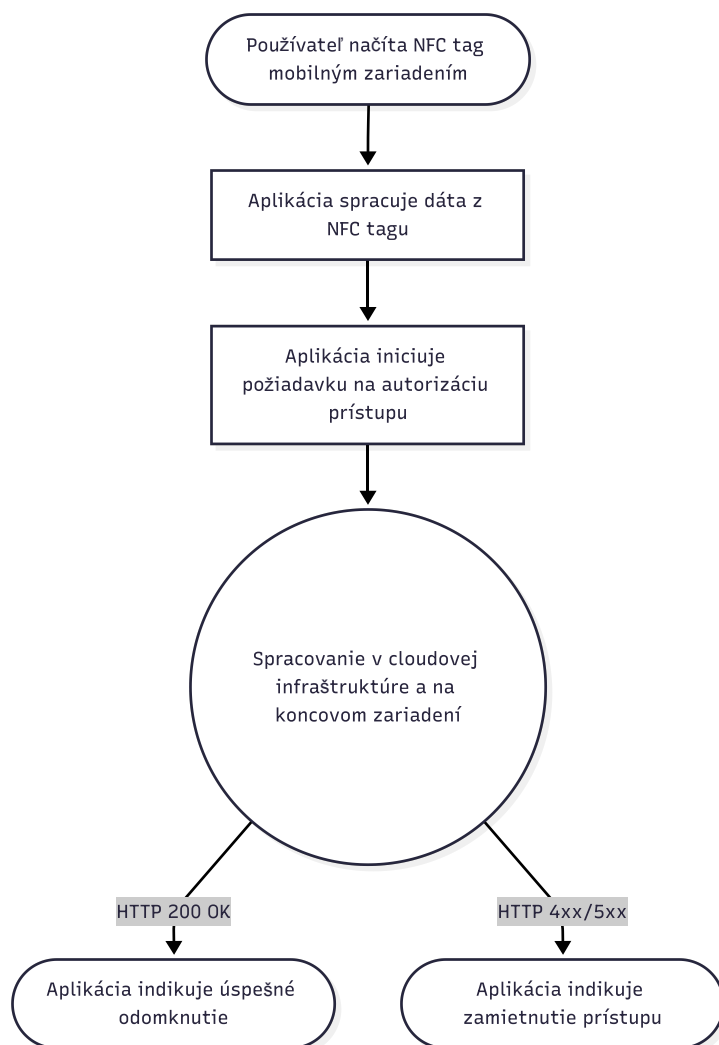
Druhá fáza, autorizačná požiadavka. Aplikácia volá Edge Function `unlock-device` s prečítaným `device_id` v tele a JWT tokenom v hlavičke `Authorization`. Požiadavka smeruje cez Cloudflare Worker proxy na Supabase. Počas čakania aplikácia zobrazuje progresívny indikátor.

Fázy tri až päť prebiehajú výhradne na serveri a sú popísané v ďalšej podsekcii.

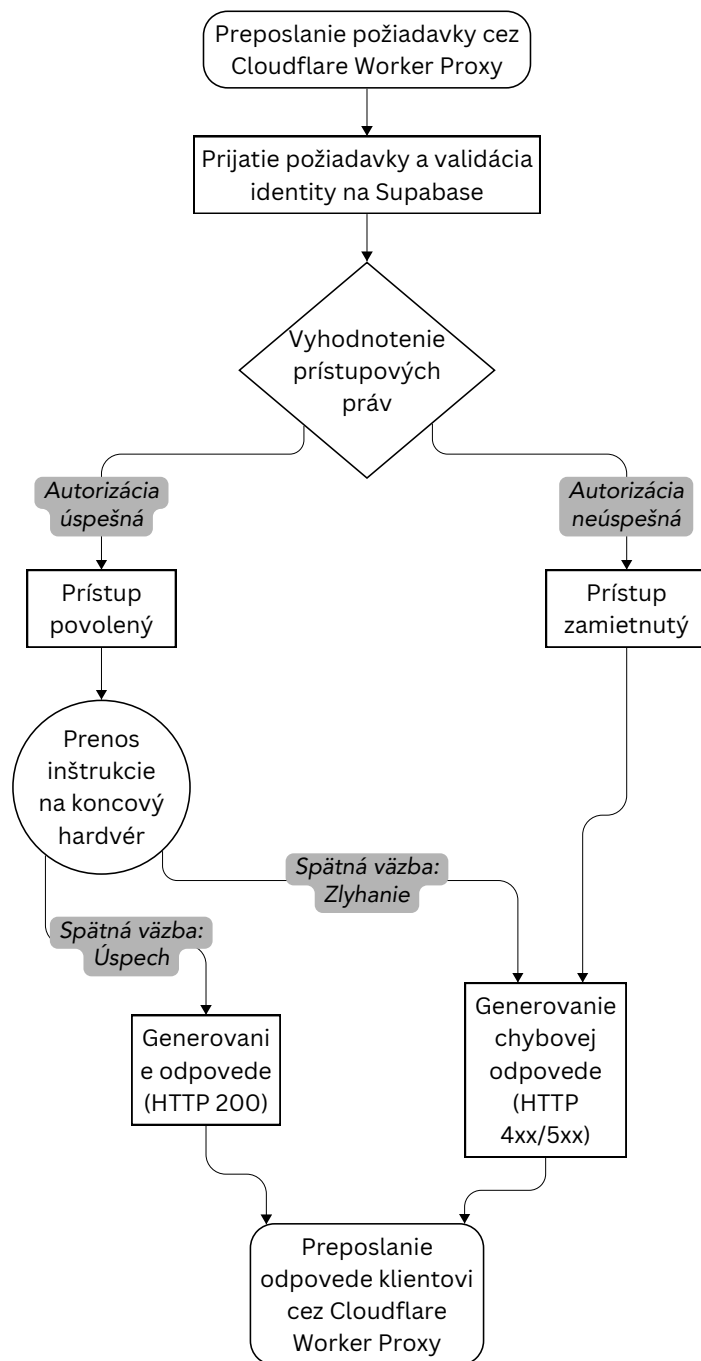
Šiesta fáza, zobrazenie výsledku. Pri HTTP 200 aplikácia zobrazí dialóg úspechu so zelenou ikonou, pri 4xx/5xx dialóg zamietnutia s červenou ikonou. Texty sú lokalizované do aktuálneho jazyka.

2.3.3 Pohľad serverovej vrstvy

V Edge Function a databáze prebieha tretia až piata fáza komunikačného toku. Konkrétne validácia JWT, vyhodnotenie prístupových práv a volanie `direct method` na Azure IoT Hub (viď diagram 2.4).



Obr. 2.3 Pohľad odomykania z perspektívy mobilnej aplikácie



Obr. 2.4 Pohľad backendovej autorizačnej logiky

Tretia fáza, validácia JWT. Edge Function po prijatí požiadavky overí autorizačnú hlavičku, ktorá dekoduje JWT a vráti identitu používateľa. Pri neplatnom alebo chýbajúcom tokene vracia HTTP 401, pri chýbajúcom identifikátore zariadenia v tele HTTP 400.

Štvrtá fáza, vyhodnotenie prístupových práv. Edge Function volá databázovú RPC funkciu napísanú v PL/pgSQL. V jedinom SQL dotaze funkcia overí príslušnosť k spoločnosti, neexistenciu logického vymazania, časovú platnosť pridelení a existenciu prístupového kľúča pokrývajúceho cieľové zariadenie. Vracia `true` pri aspoň jednej platnej kombinácii. Presunutie logiky do databázy zaručuje, že aplikácia nevykonáva žiadne autorizačné rozhodovanie a komplexný viactabuľkový dotaz prebehne v jednej transakcii.

Piata fáza, volanie `direct method`. Pri pozitívnej autorizácii Edge Function dynamicky generuje krátkodobý SAS token cez HMAC-SHA256 podpis URI a času expirácie, pričom kľúč Azure IoT Hub je v environment variables Edge Function. S týmto tokenom funkcia volá REST API Azure IoT Hub Service [28], ktorý buď synchronne odovzdá výsledok, alebo vráti chybu pri offline zariadení. Edge Function následne tvorí HTTP odpoveď pre aplikáciu.

2.3.4 Pohľad IoT zariadenia

Zámok ako koncový aktér realizuje šiestu fázu. Po doručení `direct method` príkazu spracuje názov metódy z MQTT topicu, vykoná zodpovedajúcu vizuálnu reakciu LED a publikuje odpoveď so stavovým kódom späť do Azure IoT Hub (viď diagram 2.5).

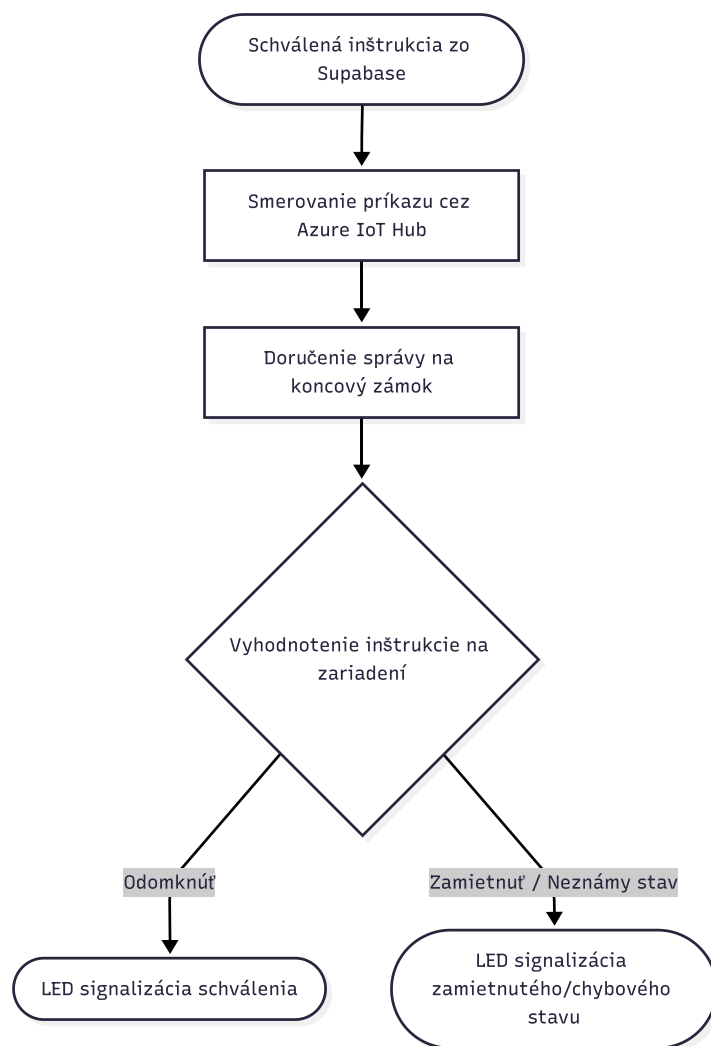
Šiesta fáza, vizuálna signalizácia. Zariadenie je trvalo pripojené k Azure IoT Hub cez MQTT na porte 8883 s TLS a odoberá predom definovaný topic [28]. Po doručení `direct method` MQTT callback vo firmvéri spracuje názov metódy z topicu a vykoná akciu, pri `unlock` sa LED rozsvieti zelenou na 3 sekundy, pri `deny` trikrát zabliká červenou, pri neznámej metóde sa nevykoná nič. Po vykonaní akcie firmvér publikuje odpoveď so stavovým kódom a zrkadleným `requestId` [28], ktorý Azure použije pre spárovanie s pôvodným volaním. MQTT spojenie zostáva otvorené pre ďalšie volania.

2.4 Bezpečnostný návrh

Bezpečnostný model systému je postavený na princípe *defense in depth* [43], kde celková bezpečnosť nie je závislá na jedinej vrstve, ale rozložená medzi päť nezávislých kontrolných bodov, transportnú bezpečnosť, autentifikáciu používateľa, autorizáciu (RLS, Edge Functions, UI), autentifikáciu komunikácie medzi servermi a princíp najmenšieho privilégia.

2.4.1 Model hrozieb

Pred popisom opatrení vymedzuje sekcia, voči akým útokom sa systém bráni. Pre kategorizáciu používame STRIDE model od Microsoftu, podrobne popísaný v knihe Shostacka [44]. Relevantné kategórie hrozieb a primárne obrany sú zhrnuté v tabuľke 2.1.



Obr. 2.5 Pohľad reakcie zámku

Kategória	Hrozba	Primárna obrana
Spoofing	Vydávanie sa za iného používateľa	JWT tokeny vystavené Supabase Auth, podpisová verifikácia pri každej operácii.
Tampering	Modifikácia chránených dát	RLS politiky v databáze, sekundárne kontroly v Edge Functions.
Information disclosure	Prístup k cudzím dátam	Multi-tenancy izolácia cez RLS politiky podľa <code>company_id</code> .
Elevation of privilege	Eskalácia administrátorských práv	Stĺpec <code>system_role</code> chránený RLS pred modifikáciou.
Repudiation	Popretie vykonanej operácie	Audit log a timestampy vo všetkých tabuľkách.
Denial of service	Znefunkčnenie služby	Cloudflare ako prvá línia obrany, rate limiting na úrovni Supabase.

Tabuľka 2.1 STRIDE kategórie hrozieb relevantné pre navrhovaný systém.

Mimo zamerania tejto práce ostáva fyzická ochrana samotného zámku pred rozobratím a ochrana pred odcudzením odomknutého telefónu s prístupom do aplikácie. Tieto problémy by si vyžadovali doplnkové opatrenia ako napríklad biometrické overenie používateľa.

2.4.2 Transportná bezpečnosť

Všetka komunikácia v systéme prebieha cez šifrované kanály. Komunikácia mobilnej aplikácie so serverovou vrstvou ide cez HTTPS s TLS 1.2 alebo novším, pričom certifikáty Supabase a Cloudflare sú vystavované dôveryhodnými certifikačnými autoritami a aplikácia neobchádza štandardnú validáciu trust store operačného systému. Komunikácia medzi Supabase Edge Function a Azure IoT Hub Service API beží nad HTTPS s TLS, obsluženým v Deno runtime. Komunikácia Azure IoT Hub a IoT zariadenia využíva MQTT nad TLS 1.2 na porte 8883, pričom Azure IoT Hub nešifrované MQTT spojenia neumožňuje [27]. Otázka overenia certifikátu Azure IoT Hub na strane ESP32 je rozobraná v sekcii implementácie.

2.4.3 Autentifikácia používateľa

Autentifikácia používateľov je realizovaná cez Supabase Auth postavený na štandarde JWT [14]. Pri prihlásení Supabase overí heslo voči bcrypt hashu v `auth.users` a vystaví pár tokenov. Krátkotrvajúci access token, ktorý sa pripája k autorizačnej hlavičke každej požiadavky, a dlhotrvajúci refresh token na obnovenie access tokenu bez opakovaneho zadávania hesla. Klient ukladá oba tokeny v bezpečnom úložisku operačného systému, pri odhlásení sú explicitne vymazané.

Pri každom volaní chráneného endpointu Edge Function dekoduje JWT, overí podpis a extrahuje identitu používateľa. Edge Function aj RLS politiky v databáze operujú nad tým istým tokenom, autentifikácia je tak konzistentná bez ohľadu na cestu, ktorou sa dotaz k dátam dostane.

2.4.4 Autorizácia

Autorizácia v systéme je realizovaná na troch úrovniach, ktoré sa vzájomne dopĺňajú. Obídenie jednej vrstvy samo o sebe nestačí na prelomenie ochrany a získanie neoprávneného prístupu.

Prvá úroveň, RLS v databáze. PostgreSQL natívne podporuje politiky RLS, ktoré sa vyhodnocujú pri každom dotaze na konkrétnu tabuľku. Politika je deklarativným pravidlom, ktoré buď dovolí, alebo zakáže prístup k jednotlivým riadkom na základe kontextu aktuálne prihláseného používateľa. Supabase integruje RLS s autentifikačným systémom cez pomocné funkcie `auth.uid()` a `auth.jwt()` [45].

Pre navrhovaný systém sú RLS politiky kľúčovým nástrojom multi-tenantnej izolácie. Konceptne sú politiky definované pre každú hlavnú tabuľku tak, aby garantovali nasledovné:

- Používateľ vidí iba dáta svojej spoločnosti.
- Používateľ vidí iba aktívne záznamy.

- Záznamy na vyššej hierarchickej úrovni sú modifikovateľné iba používateľom s vyššou systémovou rolou.

Výhoda tohto prístupu je zásadná. Aj keby aplikačná vrstva obsahovala chybu a pokúsila sa odoslať dotaz bez správneho filtru, databáza by ho vyhodnotila bezpečne, politika sa vždy bez výnimky aplikuje na každý dotaz.

Druhá úroveň, autorizácia v Edge Functions. Tam kde operácia vyžaduje logiku presahujúcu deklaratívne RLS politiky, je doplnková kontrola implementovaná v Edge Function. Typickým príkladom je samotná funkcia `unlock-device`, kde RLS neobsluží otázku *má používateľ X práve teraz právo odomknúť zariadenie Y?*, pretože tá vyžaduje traverzovanie viacerých tabuliek s vyhodnotením časovej platnosti. Preto volá RPC funkciu `can_unlock_device`, ktorá túto logiku enkapsuluje, a výsledok premieta do HTTP 200 alebo 403 odpovede.

Ďalším príkladom je Edge Function `create-new-account`, ktorá vykonáva kontrolu pred volaním Supabase Admin API. Nový účet s rolou `manager` môže vytvoriť iba používateľ s rolou `owner`, teda vlastník spoločnosti. Táto kontrola je vykonávaná explicitne v kóde funkcie.

Tretia úroveň, kontrola v UI. Najpovrchnejšou vrstvou je ovládanie viditeľnosti prvkov v mobilnej aplikácii na základe role používateľa. Bežný používateľ nevidí tlačidlá pre administratívne operácie a nemôže sa navigovať na administratívne obrazovky. Táto vrstva síce nie je bezpečnostnou v zmysle ochrany pred útokom, ale je dôležitou súčasťou použiteľnosti systému. Jej cieľom je predísť omylom a nejasnému používateľskému prostrediu.

2.4.5 Autentifikácia komunikácie medzi servermi

Medzi serverovými komponentmi prebiehajú tri druhy autentifikovaných volaní s rôznymi mechanizmami.

Edge Function a Supabase databáza. Edge Function používa jeden z dvoch Supabase kľúčov. `anon_key` s obmedzenými oprávneniami podliehajúci RLS politikám, alebo `service_role_key` obchádzajúci RLS pre administratívne operácie. Pre volania v kontexte autentifikovaného používateľa sa využíva `anon_key` s pripojeným JWT tokenom, čím zostávajú RLS politiky aktívne. `Service role key` sa používa výhradne pre administratívne operácie cez Supabase Admin API.

Edge Function a Azure IoT Hub. Autentifikácia cez SAS token, dynamicky generovaný v Edge Function pri každom volaní s krátkou platnosťou. Token je HMAC-SHA256 podpis URI a expirácie, kľúčom je primárny kľúč Azure IoT Hub prístupovej politiky s obmedzeniami na invokáciu `direct methods`.

Zariadenie a Azure IoT Hub. Zariadenie sa autentifikuje SAS tokenom špecifickým pre dané zariadenie, podpísaným primárnym kľúčom uloženým v Azure IoT Hub device registry. Kľúč je vo firmvéri uložený ako konštanta. Model poskytuje dve dôležité vlastnosti. Kompromitácia jedného zariadenia neohrozí ostatné a registry umožňuje okamžitú revokáciu konkrétneho zariadenia.

2.4.6 Princíp najmenšieho privilégia

Systém sa drží princípu najmenšieho privilégia, formalizovaného v práci Saltzera a Schroedera [46], kde každý aktér má len také oprávnenia, ktoré nevyhnutne potrebuje.

Bežný používateľ vidí iba pridelené prístupy a zariadenia v rámci svojej spoločnosti, bez prístupu k zoznamu iných používateľov či k dátam iných spoločností.

Manažér navyše spravuje zariadenia a privilégia v rámci svojich budov či oblastí.

Vlastník má plné oprávnenia v rámci svojej spoločnosti, k cudzím dátam však prístup nemá.

Systémový administrátor prevádzky v tomto modeli neexistuje ako produktová rola, administrátorské úkony nad celou platformou sa vykonávajú priamo cez Supabase admin rozhranie.

2.4.7 Bezpečnosť IoT zariadenia

Bezpečnostný model IoT zariadenia je zúžený tým, že zariadenie neobsahuje žiadnu časť autorizačnej logiky. Jeho úlohou je striktne prijať autentifikovaný direct method príkaz a vykonať vizuálnu spätnú väzbu. Z toho vyplývajú dve dizajnové vlastnosti.

Zariadenie nemá prístup k používateľským dátam. Vo firmvéri nie sú uložené žiadne používateľské tokeny, kľúče ani zoznamy oprávnených osôb. Jediný secret, ktorý zariadenie pozná, je jeho vlastný SAS token. Pri fyzickom rozobratí a získaní tokenu útočník dokáže vystupovať ako toto konkrétne zariadenie voči IoT Hub, ale nedokáže odomknúť iné zariadenia ani získať prístup k citlivým systémovým dátam.

Revokácia zariadenia je centralizovaná. Pri podozrení na kompromitáciu stačí v Azure IoT Hub device registry zneplatniť kľúč daného zariadenia, prípadne odstrániť zariadenie zo Supabase, čím prestane figurovať v odpovediach RPC funkcie `can_unlock_device`. Implementačné obmedzenia bezpečnosti firmvéru ESP32 a odporúčania pre produkčné nasadenie sú detailne rozobrané v implementačnej kapitole.

3 Implementácia

Kapitola popisuje praktickú realizáciu navrhnutého systému, od natívnych komponentov mobilnej aplikácie, cez cross-platformovú implementáciu, cloudovú infraštruktúru, až po firmvér ESP32 zámku.

3.1 Implementácia mobilnej aplikácie

Mobilná aplikácia je z pohľadu kódovej základne najrozsiahlejším komponentom systému. Je postavená nad frameworkom Flutter a prostredníctvom natívnych pluginov integruje platformovo špecifické NFC API, pričom zvyšok kódu ostáva zdieľaný medzi platformami iOS a Android.

3.1.1 Použité technológie a kľúčové koncepty

Voľba technologického stacku, Flutter ako cross-platformový framework a oficiálny Supabase Flutter klient pre komunikáciu so serverovou vrstvou, je zdôvodnená v prvej kapitole. Aplikácia ďalej využíva balíčky **Riverpod** pre stavový manažment, **GoRouter** pre navigáciu a pomocné balíčky **flutter_dotenv** pre načítavanie konfiguračných premenných, **flutter_secure_storage** pre abstrakciu nad iOS Keychain a Android EncryptedSharedPreferences a **intl** so vstavaným Flutter mechanizmom lokalizácie UI do troch jazykov.

Detaily použitia konkrétnych technológií sú rozobrané v nasledujúcich podsekcích.

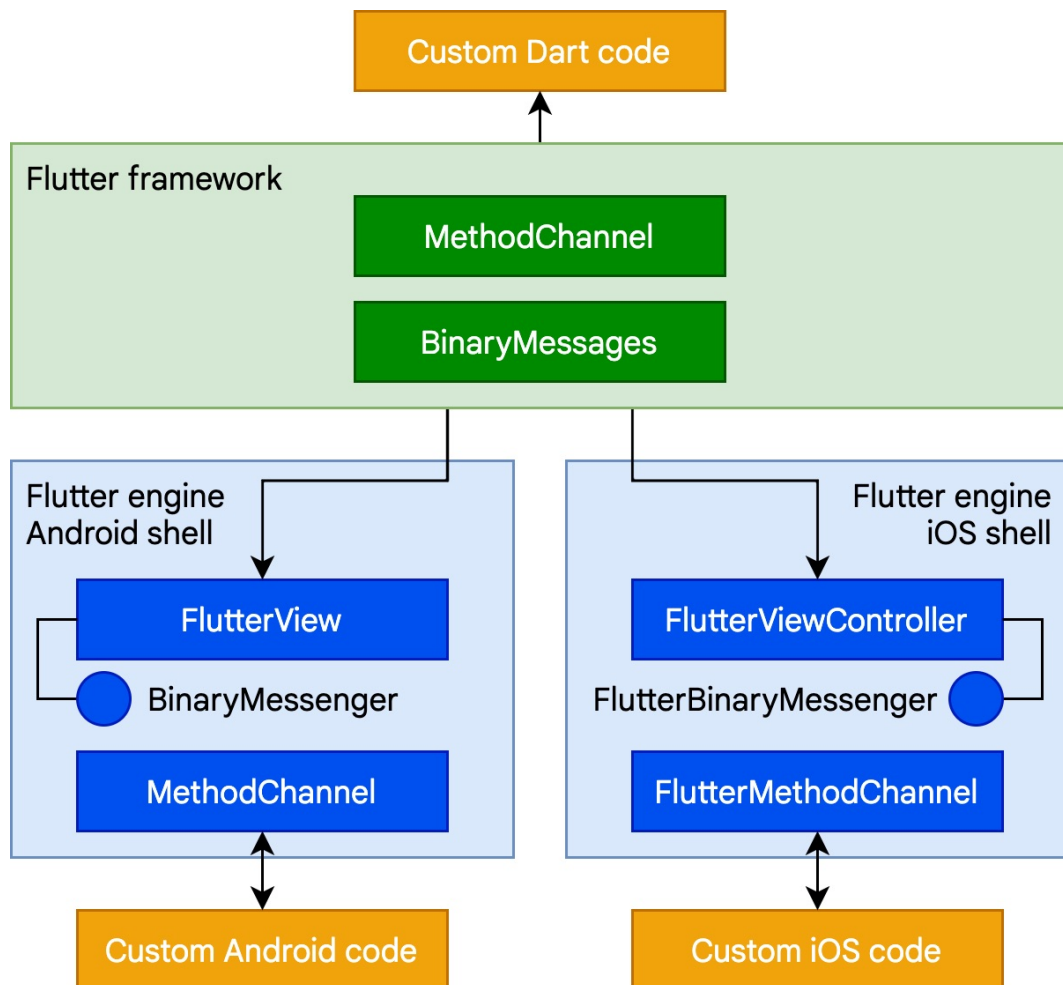
3.1.2 Integrácia natívneho kódu

NFC API patrí medzi platformovo špecifické funkcie, ktoré nie sú pokryté štandardnou Flutter abstrakciou. iOS a Android poskytujú odlišné rozhrania s odlišnou sémantikou životného cyklu, spojiť ich do unifikovaného Dart API by viedlo k strate kontroly nad detailami. Riešením je mechanizmus *platform channels*, kanál pre asynchrónne volania a výmenu serializovateľných dát medzi Dart a natívnou vrstvou [24].

V rámci tejto aplikácie je NFC funkcionality realizovaná ako samostatný Flutter plugin pomenovaný **nfc_device_manager**, ktorý enkapsuluje tri vrstvy, Dart API, natívnu implementáciu iOS v jazyku Swift a Android v jazyku Kotlin. Z pohľadu Dart kódu zvyšku aplikácie je práca s NFC rovnaká bez ohľadu na platformu, plugin interne rozhoduje, ktorú natívnu implementáciu zavolať.

Dart rozhranie pluginu vystavuje tri základné metódy.

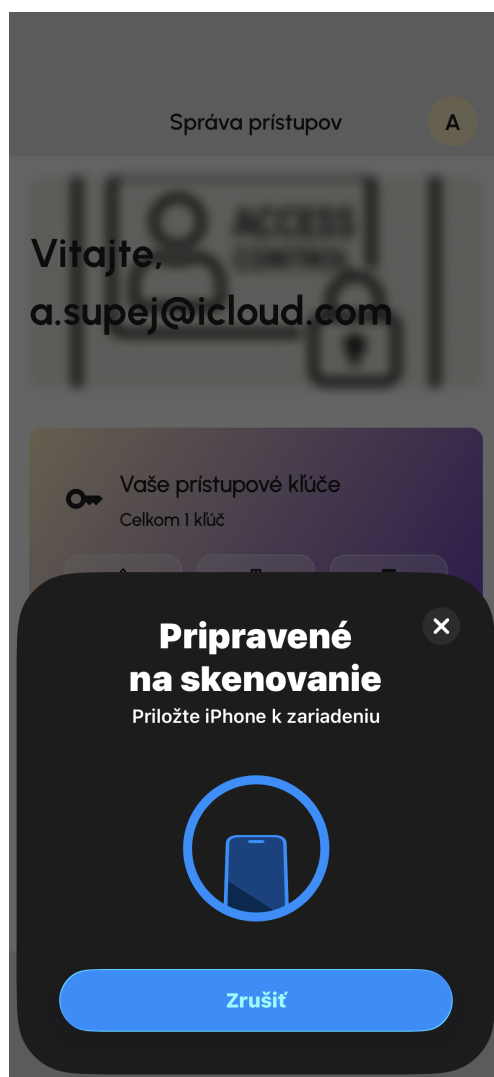
- **isNfcSupported()** zisťujúci, či zariadenie disponuje NFC hardvérom.
- **readDeviceId()** spúšťajúci samotné čítanie NFC tagu.
- **stopSession()** pre ukončenie aktuálneho čítania.



Obr. 3.1 Platform channels (zdroj: [47]).

Metódy vracajú výsledok ako asynchrónny **Future**. Interne Dart vrstva vyvolá platformové volanie cez **MethodChannel**, ktorý smeruje Flutter framework k príslušnej natívnej implementácii. Výsledkom úspešného čítania je objekt obsahujúci prečítaný reťazec. V prípade zlyhania plugin vracia chybu.

iOS implementácia pluginu je postavená nad frameworkom **CoreNFC**, dostupným od verzie iOS 11 pre čítanie a od iOS 13 pre dynamické NFC tagy typu 5. [33]. Kľúčovou vlastnosťou CoreNFC je, že systém automaticky zobrazuje natívny modálny dialóg so zvolenou inštrukčnou hláškou (viď obrázok 3.2. Tento modál na iOS jednak informuje používateľa o prebiehajúcej operácii a zároveň slúži ako bezpečnostné opatrenie zo strany systému, ktoré zabráňuje aplikáciám na pozadí tajne skenovať NFC tagy bez vedomia používateľa.



Obr. 3.2 Natívny modálny dialóg.

Implementačne je čítanie iniciované vytvorením inštancie `NFCNDEFReaderSession` s delegátom implementujúcim dva callback protokoly. `readerSession(_:didDetectNDEFs:)` pre úspešné prečítanie tagov a `readerSession(_:didInvalidateWithError:)` pre všetky zlyhania. Po priložení tagu iOS zavolá prvý callback s počtom prečítaných NDEF správ, z ktorých sa vyberie prvý záznam typu *Text record*.

Android implementácia pluginu využíva systémové API **NfcAdapter** dostupné od API level 10. Architektúra Androidu pre NFC je od iOS zásadne odlišná. Neexistuje koncept modálneho dialógu iniciovaného aplikáciou, namiesto toho sa aplikácia musí zaregistrovať ako príjemca zámerov NFC cez mechanizmus *foreground dispatch*. Tento mechanizmus znamená, že keď je aktivita aplikácie na popredí a používateľ priloží telefón k tagu, systém automaticky doručí aktivite obsah prečítaných dát z tagu cez callback `onNewIntent`.

Implementačne plugin pri volaní `readDeviceId()` najprv aktivuje foreground dispatch s filtrom na `NDEF_DISCOVERED` zámery a filtrom MIME typu `text/plain`, čo zabezpečí, že aplikácii budú doručené iba tagy obsahujúce textové NDEF správy. Aplikácia si otvorí čakajúcu promise pre výsledok a čaká na doručenie. Po priložení tagu je v `onNewIntent` prečítaný `NDEF_MESSAGES`, z neho vybraný prvý záznam.

Parovanie NDEF Text Record payloadu. Po získaní prvého NDEF záznamu obe natívne implementácie aplikujú totožnú logiku spracovania podľa špecifikácie NFC fóra [48]. Payload textového záznamu pozostáva zo status bajtu, jazykového identifikátora a samotného textu, v našom prípade UUID zariadenia. Prvý bajt sa interpretuje ako status bajt, z ktorého dolných šesť bitov určí skutočnú dĺžku jazykového identifikátora. Reťazec UUID sa vyjme od pozície za jazykovým identifikátorom do konca payloadu. Tento dynamický prístup zabezpečuje korektné spracovanie záznamov bez ohľadu na použitý jazykový kód, ktorý môže mať dva alebo tri znaky.

Zjednotenie z pohľadu volajúceho kódu. Napriek odlišným natívnym implementáciám je Dart API pluginu identické a zvyšok aplikácie s ním pracuje bez znalosti platformových detailov. Tým je platformovo špecifický kód oddelený do úzkej natívnej vrstvy s jasne definovaným rozhraním, pričom hlavná biznis logika sa zdieľa medzi platformami.

3.1.3 Architektúra a správa stavu aplikácie

Aplikácia je štruktúrovaná podľa princípu Single Responsibility Principle, kde každá vrstva má jasne definovanú úlohu a interakcie medzi vrstvami prebiehajú cez explicitné rozhrania. Tento prístup zlepšuje následnú testovateľnosť, uľahčuje orientáciu v kóde pri jeho neskoršom rozvoji a redukuje riziko, že zmena v jednej časti aplikácie spôsobí neočakávané dôsledky v inej časti. Táto podsekcia popisuje organizáciu projektu, dátovú vrstvu a prístup k správe stavu pomocou balíčka Riverpod.

Organizácia projektového stromu

Zdrojový kód aplikácie je umiestnený v adresári `lib/` a je rozdelený do týchto hlavných sekcií.

- `lib/core/`. Infraštruktúrne komponenty využívané naprieč celou aplikáciou, ako napríklad klient Supabase, NFC plugin, práca so Secure Storage zariadenia alebo načítavanie konfigurácie z `.env` súborov a definícia navigačného routera.
- `lib/models/`. Triedy reprezentujúce entity z dátového modelu, či už ide o spoločnosti, budovy, zariadenia alebo účty a ich prístupové kľúče. Každá

trieda obsahuje factory konštruktory pre konverziu z JSON a metódy `toJson()` pre opačný prípad, plus pomocné funkcie.

- **lib/features/**. Funkčné moduly aplikácie. Každá feature v tomto adresári reprezentuje samostatný funkčný celok, či už ide o celú obrazovku spolu s jej vlastnými widgetmi a providermi, alebo samostatne znovupoužiteľný widget volaný z viacerých častí aplikácie. V prvom prípade obsahuje feature typicky jeden top-level widget obrazovky, pomocné widgety použité výhradne na tejto obrazovke a Riverpod providery spravujúce jej stav. V druhom prípade feature obsahuje iba samotný komponent a jeho závislosti.

Toto členenie dodržiava konvenciu *feature-first*, kde všetky súbory jednej funkcionality sú pohromade v jednom adresári namiesto roztrúsenia podľa technickej vrstvy. Výhodou je lepšia lokálnosť zmien a hranica feature priečinka ako prirodzený modulárny kontrakt, kde závislosti medzi features sú viditeľné na úrovni importov.

Dátová vrstva

Centrálnym bodom komunikácie so serverovou vrstvou je trieda **SupabaseService**, implementovaná pomocou návrhového vzoru singleton. Dôvodom voľby singletonu je, že Supabase klient interne spravuje dlhotrvajúce spojenie s backendom vrátane automatickej obnovy autentifikačných tokenov, bez potreby udržiavať viacero paralelných inštancií klienta pre tú istú aplikáciu.

SupabaseService zapuzdruje všetky operácie nad databázou do typovo bezpečných metód, ktoré vracajú doménové objekty namiesto surových JSON máp. Typicky ide o metódy ako `fetchCurrentUserAccessKeys()`, `createBuilding()`, `updateDevice()` a podobne. Každá metóda interne buduje dotaz cez Supabase fluent API, spracuje odpoveď a vráti deserializovaný výsledok. Chybové stavy sú transformované na typizované výnimky, ktoré sa prenechajú réžii volajúcej funkcie.

Správa stavu

Aplikácia využíva na správu interného stavu balíček Riverpod ako jediný mechanizmus pre zdieľanie stavu medzi obrazovkami a pre reaktívne prepojenie UI s dátami.

Infraštruktúrne providery. Najnižšia vrstva provideroz poskytuje prístup k základným službám ako je napríklad prístup k Supabase, prístup k NFC manažéru alebo k Secure Storage.

Autentifikačné providery. Táto vrstva riadi životný cyklus používateľskej relácie. **authNotifierProvider** spravuje stavový automat autentifikácie a poskytuje metódy na prihlásenie, odhlásenie alebo obnovenie relácie. Pri volaní metódy prihlásenia sa prechod cez stavy volá automaticky. Počas prihlasovania sa prepne do stavu **loading** a po úspechu do **authenticated** s načítaným objektom účtu. Jednotlivé UI komponenty sledujú tento stav cez `ref.watch(authNotifierProvider)` a reaktívne zobrazujú stav.

Taktiež sú v aplikácii prítomné dva pomocné providery, ktoré vracajú jednoduché hodnoty. **isAuthenticatedProvider** vracia `true/false` podľa stavu

prihlásenia a `currentUserProvider` vráti objekt aktuálne prihláseného používateľa.

Dátové providery reaktívne načítavajú dáta zo Supabase, prevažne typu `FutureProvider`. Konkrétne sú v aplikácii inštalácie `currentUserAccessKeysProvider` (zoznam pridelených kľúčov pre prihláseného používateľa), `companyHierarchyProvider` (kompletná hierarchia spoločnosti pre administratívne obrazovky) a `deviceDetailProvider.family(deviceId)` (detail konkrétneho zariadenia). Každý interne volá príslušnú metódu `SupabaseService` a vracia `AsyncValue<T>`. Riverpod automaticky reťazí závislosti, pri zmene hodnoty providera sú všetky závislé providery zneplatnené a ich cache zahodená. Po úspešnej zmene dát na serveri aplikácia volá `ref.invalidate(providerName)`, čo vynúti znovunačítanie zo zdroja a UI sa automaticky aktualizuje.

3.1.4 Bezpečné ukladanie používateľských tokenov

Po prihlásení získa klient pár tokenov (krátkotrvajúci access token a dlhotrvajúci refresh token) [49]. Bez perzistentného úložiska by používateľ musel zadávať heslo pri každom štarte aplikácie. Naivné úložisko v `SharedPreferences` alebo `UserDefaults` by však znamenalo triviálnu získateľnosť tokenov pri kompromitácii zariadenia. Supabase Flutter klient umožňuje injektovať vlastnú implementáciu úložiska cez rozhranie `LocalStorage`, čím zveruje aplikácii zodpovednosť za bezpečnosť. Aplikácia implementuje toto rozhranie v triede `SecureStorageService`, ktorá deleguje na balíček `flutter_secure_storage`. Balíček poskytuje platformovo špecifickú abstrakciu nad systémovými bezpečnostnými úložiskami, na iOS **Keychain** a na Androide **EncryptedSharedPreferences** z knižnice `AndroidX Security`. Obe úložiská poskytujú hardvérovo podporované šifrovanie tokenov. iOS Keychain používa AES-256-GCM s kľúčmi chránenými dedikovaným hardvérovým koprocesorom Secure Enclave, ktorý má vlastnú spúšťaciu ROM, AES engine a izolovanú pamäť [50, 51]. Android EncryptedSharedPreferences používa AES-256 v režime GCM s master kľúčom v Android Keystore, ktorý je umiestnený v Trusted Execution Environment alebo v hardvérovom StrongBox čipe [52]. Detailné kryptografické mechanizmy sú nad rámec tejto práce, podstatné je, že obe riešenia predstavujú odporúčané platformové úložisko od výrobcov a poskytujú izoláciu od iných aplikácií.

3.1.5 Správa vývojových prostredí

Aplikácia je navrhnutá tak, aby bola zostaviteľná v dvoch oddelených prostrediach, určených na vývoj a produkciu. Každé prostredie má vlastnú kombináciu backendových endpointov, konfiguračných premenných a metadát inštalateľného balíčka, čo umožňuje paralelnú prevádzku oboch verzií aplikácie na rovnakom fyzickom zariadení bez konfliktov.

Flutter Flavors

Flutter podporuje koncept *flavors*, zjednocujúci pojem pre platformovo špecifické variantové buildy [53]. Na Androide flavor zodpovedá *product flavor* v Gradle, na iOS *build configuration* a *scheme* v Xcode. Flutter SDK pri `flutter build --flavor` medzi nimi prepína.

V návrhu aplikácie sú definované dve flavors **dev** a **prod**. Každý z nich má nasledujúce parametre.

- **Odlišný bundle identifier.** Android product flavor pridáva suffix **.dev** k základnému identifikátoru aplikácie a iOS scheme priraduje samostatný identifikátor balíčka. Tým je systémom garantované, že obe varianty sú samostatné aplikácie a môžu byť naraz nainštalované na rovnakom zariadení bez toho, aby sa prepisovali.
- **Odlišný názov aplikácie.** Aplikácia určená pre vývojové prostredie má k názvu prilepený suffix **dev**. Vývojár okamžite vidí, ktorú verziu na domovskej obrazovke otvára.
- **Odlišnú ikonu aplikácie.** Odlíšená ikona pre dev flavor pomáha predísť zámene, podobne ako pri názve.

Flutter dokumentácia odporúča pre každý flavor vytvoriť samostatný vstupný bod v podobe `main_<flavor>.dart` súboru, ktorý obsahuje minimum kódu a deleguje na spoločný inicializačný kód [53]. Navrhovaný projekt tento vzor aplikuje a sú implementované súbory `main_dev.dart` a `main_prod.dart`, každý z nich načíta príslušný `.env` súbor a následne zavolá spoločnú funkciu, v ktorej prebehne inicializácia Supabase klienta, nastavenie Secure Storage komponenty a samotné spustenie koreňového widgetu.

Konfiguračné premenné

Pre oddelenie vlastnej konfigurácie od kompilačných variantov využíva aplikácia balíček `flutter_dotenv` [54], ktorý umožňuje načítavať runtime konfiguračné premenné zo súborov vo formáte `.env`. Aplikácia má dvojicu takýchto súborov opäť so zameraním na konkrétne vývojové prostredie. Medzi kľúčové konfiguračné premenné patria nasledujúce.

- `SUPABASE_URL`, URL adresa Supabase projektu, ktorá sa môže líšiť medzi prostrediami na separáciu produkčným a vývojových dát.
- `SUPABASE_ANON_KEY`, verejný anonymný kľúč pre komunikáciu so Supabase
- `CLOUDFLARE_WORKER_URL`, URL adresa nasadeného Cloudflare Worker proxy
- `USE_CLOUDFLARE`, jednoduchý prepínač medzi priamym volaním Supabase alebo smerovaním cez Cloudflare.

Dokumentácia balíčka `flutter_dotenv` upozorňuje, že po kompilácii sú všetky hodnoty prítomné v inštaláčnom balíčku a extrahovateľné reverzným inžinierstvom. Aplikácia tento limit rešpektuje a v `.env` súboroch ukladá iba verejné identifikátory a feature flags, všetky tajné a produkčné heslá sú na serverovej strane. Súbor `.env.example` je verzovaný v repozitári, samotný `.env` je vylúčený cez `.gitignore` a distribuuje sa oddelene.

Spúšťanie a build

Zostavenie a spustenie konkrétneho flavoru je otázkou jedného argumentu v príkazovom riadku.

- `flutter run --flavor dev -t lib/main_dev.dart`,
build určený pre vývoj spustený v debug móde.
- `flutter run --release --flavor prod -t lib/main_prod.dart`,
produkčný build spustený v release móde.
- `flutter build apk --flavor prod -t lib/main_prod.dart`,
produkčný release build pre Android.
- `flutter build ipa --flavor prod -t lib/main_prod.dart`,
produkčný release build pre iOS.

Argument `--flavor` vyberá platformovú konfiguráciu, `-t` definuje vstupný bod pre Dart [53]. Pre zjednodušenie vývoja sú v IDE nastavené launch konfigurácie umožňujúce spustenie flavoru jedným kliknutím.

3.1.6 Implementácia používateľského rozhrania

Používateľské rozhranie je vrstva, v ktorej sa všetky predchádzajúce technické rozhodnutia stretávajú a pretavujú do konkrétnych obrazoviek.

Navigácia s GoRouter a ochrana ciest

Navigácia v rámci aplikácie je realizovaná cez balíček **GoRouter**, deklaratívny router. V centrálnom konfiguračnom súbore je nadefinovaný zoznam ciest typu `GoRoute`, každá s URL vzorom a builder funkciou, ktorá vracia widget cieľovej obrazovky. Celý router je vystavený ako Riverpod provider, čím sa dá znovu inicializovať pri zmene závislostí a integruje sa s ostatnou stavovou vrstvou aplikácie.

Route guard cez redirect callback. GoRouter poskytuje mechanizmus `redirect`, ktorý sa volá pri každej navigácii a umožňuje vrátiť alternatívnu cieľovú URL alebo `null` pre povolenie pôvodnej navigácie [55]. Aplikácia tento mechanizmus využíva ako centralizovaný bod kontroly oprávnení. V jednej funkcii sú zhromaždené tri pravidlá, ktoré sa konzistentne uplatňujú naprieč všetkými obrazovkami.

- Neautentifikovaný používateľ smerujúci kamkoľvek inam než na `/auth` je presmerovaný na prihlasovaciu obrazovku.
- Autentifikovaný používateľ otvárajúci `/auth` je presmerovaný na domovskú obrazovku.
- Používateľ so systémovou rolou `user` smerujúci na administratívne cesty je presmerovaný na domovskú obrazovku, keďže tieto sekcie sú vyhradené iba pre správcov systému.

Reaktivita voči zmene autentifikačného stavu. GoRouter ďalej poskytuje parameter `refreshListenable`, objekt, pri ktorého notifikácii router okamžite znovu vyhodnocuje aktuálnu cestu voči `redirect callbacku` [55]. Aplikácia vytvára jednoduchý `GoRouterRefreshStream` adaptér, ktorý počúva na `isAuthenticatedProvider` a `currentUserProvider` z Riverpodu a zmenu ich hodnoty premieta do notifikácie závislých widgetov. Dôsledkom je, že pri odhlásení používateľa, router automaticky vyhodnotí aktuálnu cestu a presmeruje používateľa na `/auth` bez potreby explicitného volania navigácie.

Táto architektúra zbavuje obrazovky zodpovednosti za kontrolu prístupu. Z bezpečnostného hľadiska ide o UX opatrenie, prípadné obídenie route guard zamedzí backend.

Prehľad kľúčových obrazoviek

Aplikácia pozostáva zo siedmich obrazoviek organizovaných do feature modulov.

Prihlasovacia obrazovka (`/auth`) je jediná dostupná neautentifikovanému používateľovi. Obsahuje formulár pre e-mail a heslo, pri odoslaní volá metódu prihlásenia na autentifikačnom provideri, počas procesu zobrazuje indikátor a v prípade chyby chybovú hlášku.

Domovská obrazovka (`/`) je centrálnym bodom aplikácie. Obsahuje uvítací prvok, štatistický prehľad počtu oblastí, budov a zariadení s prístupom a dominantné tlačidlo `TapToScan` na odomykanie (viď obrázok 3.4).

Obrazovka prístupových kľúčov (`/access_keys`) zobrazuje detailný zoznam pridelených kľúčov s vyhľadávaním. Po rozkliknutí záznamu sa zobrazí zoznam zariadení patriacich do daného kľúča.

Administratívne obrazovky (`/accounts`, `accesses`, `company`) sú dostupné len pre správcovské role. Spravujú používateľské účty (cez Edge Function `create-new-account`), prístupové kľúče s časovým ohraničením a kompletnú hierarchickú štruktúru organizácie cez tabovanú navigáciu (viď obrázok 3.3 pre obrazovku Správy spoločnosti so záložkou budovy). Pri správe zariadení je dostupná aj funkcia hromadného vytvorenia s prefixom a automatickým číslovaním.

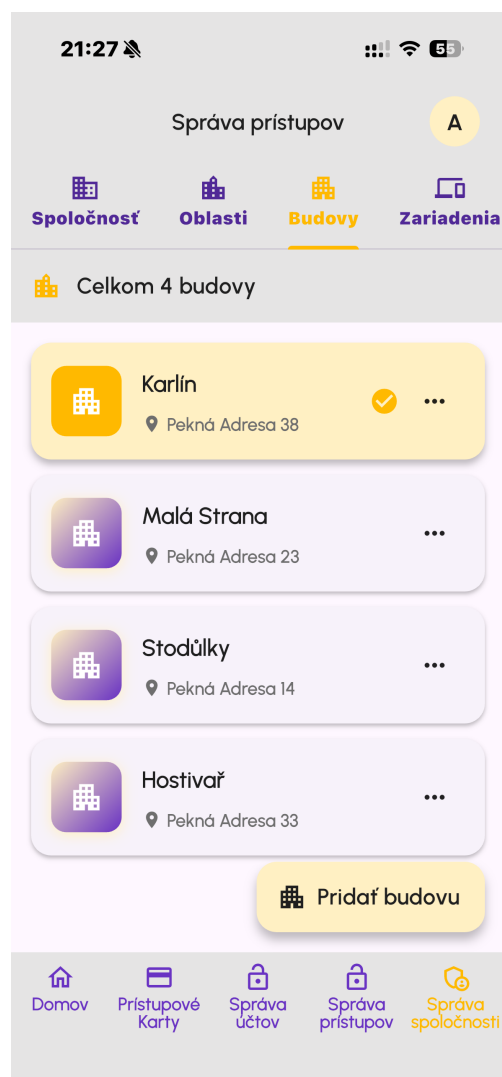
Obrazovka profilu (`/profile`) zobrazuje informácie o prihlásenom používateľovi, štatistiky spoločnosti a tlačidlo na odhlásenie.

Widget TapToScan

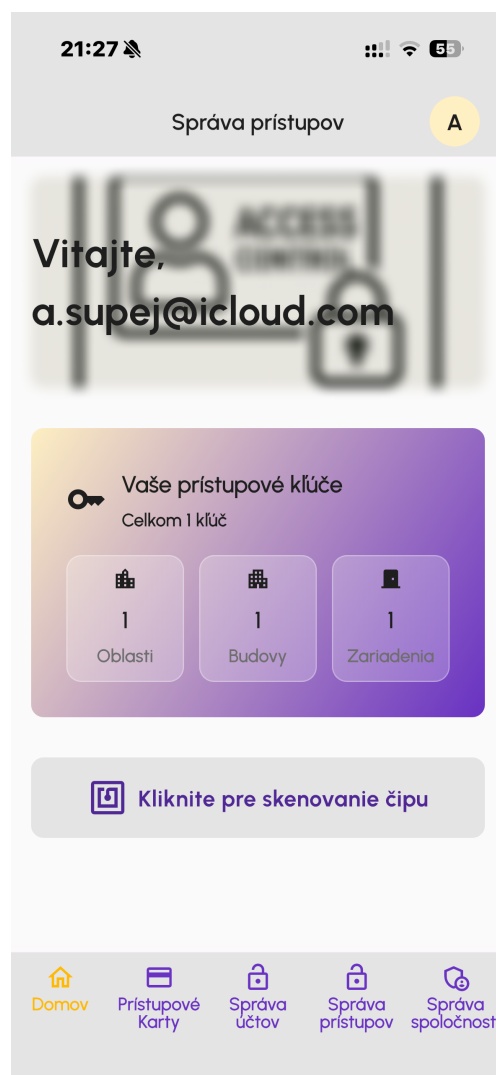
Widget `TapToScan` na domovskej obrazovke orchestruje celý klientský odomykací tok. Implementačne ide o `GestureDetector` s veľkou vizuálnou oblasťou. Po stlačení widget volá `NfcDeviceManager.readDeviceId()` (na iOS zobrazí systémový modál, na Androide aktivuje `foreground dispatch`), po prečítaní UUID volá `SupabaseService.unlockDevice(deviceId)`, ktorá interne volá Edge Function `unlock-device`, a podľa HTTP odpovede zobrazí dialóg úspechu alebo chyby.

Lokalizácia do troch jazykov

Aplikácia natívne podporuje tri jazyky. Slovenčinu ako predvolený jazyk, češtinu a angličtinu. Lokalizácia je realizovaná oficiálnym Flutter mechanizmom postaveným na balíčkoch `flutter_localizations` a `intl` [56].



Obr. 3.3 Administratívna obrazovka správy spoločnosti so záložkou budovy.



Obr. 3.4 Domovská obrazovka aplikácie.

ARB súbory. Textové reťazce sú uložené v súboroch formátu ARB, ide o formát založený na JSON, ktorý okrem samotných prekladov podporuje aj metadáta [56]. V projekte sa nachádzajú tri `.arb` súbory, umiestnené v adresári `lib/l10n`.

Generovanie triedy `AppLocalizations`. Pri spustení príkazu `flutter gen-l10n` alebo automaticky počas `flutter pub get` Flutter vygeneruje z `.arb` súborov typovo bezpečnú triedu `AppLocalizations` s getter metódou pre každý zaregistrovaný kľúč [56]. Vďaka tomuto prístupu widgety pristupujú k prekladom cez notáciu `AppLocalizations.of(context).title`. Správnosť súborov, premenných a kontrola preklepov sa deje už pri kompilácii.

Registrácia v `MaterialApp`. V koreni stromu widgetov je `MaterialApp.router` nakonfigurovaný s `localizationsDelegates` (vrátane `AppLocalizations.delegate`) a `supportedLocales` pre tri lokalizácie [56]. Flutter pri štarte automaticky vyberie jazyk podľa nastavenia operačného systému, pri nepodporovanom jazyku použije angličtinu ako fallback.

3.2 Backendová a cloudová vrstva

Backendová vrstva je distribuovaná medzi dve oddelené cloudové infraštruktúry, Cloudflare a Supabase, pričom Supabase je prepojená s Azure IoT Hub pre komunikáciu so zariadením.

3.2.1 Cloudflare Worker ako proxy

Motivácia pre Cloudflare Worker je popísaná v druhej kapitole. Implementačne ide o serverless platformu nad V8 JavaScript runtime, kde je každý worker exportovaný ako ES modul s metódou `fetch(request, env, ctx)` [57]. Worker v nej vykonáva jednoduchý URL prepis, zamení cieľovú adresu za Supabase endpoint a hlavičky zachová bez zmeny. Kód má v praxi rozsah niekoľkých desiatok riadkov.

Worker bol nasadený cez webové rozhranie Cloudflare dashboardu a po nasadení je dostupný zo všetkých dátových centier globálnej siete [57], čím je latencia v praxi v rádoch desiatok milisekúnd.

3.2.2 Databáza a Edge Functions

Supabase Edge Functions sú serverless TypeScript funkcie nad Deno runtime distribuovaným v Supabase Edge infraštruktúre [58]. Každá funkcia je `.ts` súbor exportujúci handler prijímajúci `Request` a vracajúci `Response`. Prichádzajúce požiadavky cez edge gateway, ktorá spracuje autentifikačné hlavičky, validuje JWT a aplikuje routovanie pred spustením funkcie [58]. Časy *studených štartov* sú v rádoch milisekúnd vďaka kompaktnému formátu ESZip [59].

Edge Function `unlock-device`

Funkcia je jadrom odomykacieho toku a je navrhnutá ako krátkodobá stateless operácia. Autentifikuje volajúceho cez JWT, validuje `device_id` v tele požiadavky, volá databázovú RPC `can_unlock_device(...)`, vygeneruje krátkodobý SAS

token a odošle direct method na Azure IoT Hub Service API. Výsledok od Azure sa transformuje na HTTP odpoveď klientskej aplikácii.

Edge Function create-new-account

V Supabase architektúre je používateľská identita vedená v dvoch tabuľkách, `auth.users` spravovaná Supabase Auth a aplikačná `public.accounts`. Funkcia orchestruje tri kroky, overenie oprávnenia volajúceho cez JWT, vytvorenie záznamu v `auth.users` s e-mailom a dočasným heslom (návratová hodnota obsahuje UUID), a vytvorenie záznamu v `public.accounts` so zhodným UUID, systémovou rolou a `company_id` prevzatým z kontextu volajúceho.

Edge Functions pre správu zariadení

Vytváranie zariadení je realizované cez dve Edge Functions, `create-iot-device` pre jedno zariadenie a `bulk-create-iot-devices` pre hromadné vytvorenie viacerých zariadení naraz. Obe riešia charakteristický distribuovaný problém systému, kde nové zariadenie musí byť registrované v dvoch nezávislých cloudových systémoch súčasne (Supabase pre kontext a privilégia, Azure IoT Hub pre MQTT pripojenie a autentifikáciu). Distribuovanú registráciu nie je možné zabezpečiť atomicky v transakčnom zmysle, operácia preto prebieha v dvoch krokoch a pri zlyhaní druhého kroku sa prvý anuluje.

Spoločná zásada oboch funkcií. Funkcia overí JWT, vloží záznam do databázy s vygenerovaným UUID, následne so SAS tokenom zaregistruje zariadenie v Azure na `/devices/{device_id}`. Pri zlyhaní Azure volania funkcia odstráni nový databázový záznam, čím sa zamedzí výskyt osirotených záznamov.

Hromadné vytvorenie. Funkcia umožňuje špecifikovať počet zariadení so spoločným prefixom a automatickým číslovaním. Vkladanie do databázy je realizované jedným `.insert` obaleným v transakcii, takže prejdú buď všetky záznamy, alebo žiadne. Po paralelnej registrácii v Azure sa skontrolujú výsledky a osirotené záznamy sa odstraňujú.

Databázové RPC funkcie

Okrem Edge Functions sú v systéme využité **PostgreSQL RPC** funkcie v jazyku PL/pgSQL. Ide o procedurálne rozšírenie SQL umožňujúce zapuzdriť komplexnú logiku priamo do databázy. Supabase tento vzor aktívne podporuje a PostgREST automaticky obaľuje RPC volania do transakcií [60]. V systéme sú implementované dve kategórie RPC funkcií.

Autorizačná logika. Funkcia `can_unlock_device` je vyhodnocovaná pri každom pokuse o odomknutie. V jedinom SQL dotaze overí príslušnosť k spoločnosti, neexistenciu logického vymazania, časovú platnosť pridelení a existenciu prístupového kľúča pokrývajúceho cieľové zariadenie.

Štatistické funkcie pre dashboard. Domovská obrazovka a obrazovka profilu zobrazujú štatistické prehľady. Tieto agregácie sú zapuzdrené do RPC funkcií, ktoré v jednom volaní vrátia štruktúrovaný JSON s predpočítanými číslami, čím sa znižuje počet dotazov zo strany klienta.

3.3 Implementácia hardvéru a IoT komunikácie

Firmvér koncového zariadenia je z pohľadu objemu kódu najmenší komponent systému, no funkčne kritický. Premieňa digitálny príkaz z cloudu na fyzickú reakciu zámku. Softvér je napísaný v Arduino dialekte C++ a beží na platforme SparkFun IoT RedBoard ESP32, ktorá komunikuje so SparkFun Qwiic Dynamic NFC/RFID Tag nad čipom ST25DV64KC cez I2C. Firmvér má dve zodpovednosti, inicializáciu NFC tagu s identitou zariadenia a kontinuálnu obojsmernú komunikáciu s Azure IoT Hub.

3.3.1 Komunikácia s dynamickým NFC tagom

Pri zapnutí zariadenie zapíše do NFC tagu svoj identifikátor v NDEF formáte čitateľnom mobilnou aplikáciou. EEPROM pamäť ST25DV64KC si obsah zachováva aj po odpojení napájania, takže by stačil jednorázový zápis pri prvom spustení [39], no v aktuálnej implementácii sa zápis vykonáva pri každom štarte. Pre testovacie zariadenie má tento prístup zanedbateľný dopad.

I2C inicializácia. Komunikácia medzi ESP32 a dynamickým tagom prebieha cez Qwiic konektor, ktorý implementuje štandardnú I2C zbernicu. Vo firmvéri je zbernica inicializovaná volaním `Wire.begin()` a následne je inštancia triedy `SFE_ST25DV64KC`, ktorá je preddefinovaná zo SparkFun Arduino knižnice pre tento model, inicializovaná volaním `nfcTag.begin()`, ktoré overí prítomnosť čipu na zbernici a prečíta jeho identifikačný register.

Zápis Capability Container. Každý NFC tag musí obsahovať takzvaný *Capability Container*. Ide o hlavičku, ktorá čítajúcemu zariadeniu oznamuje, aký typ tagu to je, akú má kapacitu a či obsahuje NDEF dáta. Knižnica SparkFun poskytuje pomocnú metódu `writeCCFile8Byte()`, ktorá zapíše štandardný osem bajtový kontajner pre dynamické NFC tagy typu 5 [39]. Tento krok sa vykoná pri inicializácii zariadenia.

Zápis UUID ako NDEF textový záznam. Identifikátor zariadenia (36-znakový UUID, zhodný s Azure Device ID a primárnym kľúčom v tabuľke zariadení v Supabase) je do tagu zapísaný vo formáte NDEF, štandardnom binárnom formáte NFC Fóra. Záznam typu T pozostáva zo status bajtu, jazykového identifikátora a samotného textu [48]. Firmvér využíva preddefinovanú metódu zo SparkFun knižnice, ktorá zostaví kompletný NDEF záznam aj s hlavičkou a zapíše ho do EEPROM.

3.3.2 Integrácia s Azure IoT Hub cez MQTT

Po inicializácii NFC tagu sa firmvér pripojí k Wi-Fi sieti a následne nadviaže trvalé MQTT spojenie s Azure IoT Hub. Toto spojenie je udržiavané po celú dobu behu zariadenia, prostredníctvom ktorého prichádzajú direct method volania z Edge Function.

MQTT klient PubSubClient. Pre MQTT protokol v Arduino prostredí sa v projekte používa knižnica **PubSubClient**. Táto knižnica implementuje MQTT 3.1.1 s podporou QoS 0 pre publikovanie a QoS 0 alebo 1 pre odber [61]. Klient vo firmvéri konfigurovaný s upraveným veľkostným limitom paketov na 2048 B, pretože predvolená hodnota 256 B je pre Azure IoT Hub volania nedostatočná.

Pripojenie cez TLS na porte 8883. Microsoft dokumentácia pre Azure IoT Hub explicitne stanovuje, že všetka MQTT komunikácia musí byť zabezpečená cez TLS a nešifrované spojenie cez port 1883 nie je podporované [27]. Zariadenie sa preto pripája na port 8883 s TLS 1.2. Vo firmvéri je TLS spojenie nadviazané cez `WiFiClientSecure`, čo je trieda poskytujúca TLS klienta kompatibilného s `PubSubClient` rozhraním.

Autentifikácie cez SAS token. Podľa oficiálnej Azure dokumentácie sa zariadenie autentifikuje voči IoT Hub cez SAS token, ktorý sa odovzdáva ako MQTT heslo. Token je HMAC-SHA256 podpis pozostávajúci z URI zariadenia a času expirácie, pričom ako kľúč slúži symetrický kľúč zariadenia registrovaného v Azure IoT Hub device registry. Vo firmvéri je tento token uložený ako konštanta v separátnom hlavičkovom súbore.

Subskripcia na direct method volania. Po pripojení sa zariadenie prihlási na odber `$iothub/methods/POST/#`, čo je system topic Azure IoT Hub pre doručovanie direct methods, kde wildcard `#` zachytáva všetky volania bez ohľadu na názov [27].

Príjem a odpoveď na direct method. Pri prijatí MQTT správy `PubSubClient` zavolá callback funkciu, ktorá z topicu vytiahne názov metódy a identifikátor požiadavky. Po vykonaní fyzickej akcie firmvér publikuje odpoveď s JSON payloadom a stavovým kódom. Identifikátor požiadavky je zrkadlený z pôvodnej správy, čím Azure IoT Hub spáruje odpoveď s pôvodným volaním a sprostredkuje ju Edge Function.

LED signalizácia stavu zariadenia. Firmvér využíva zabudovanú WS2812 RGB LED diódu ovládanú cez knižnicu Adafruit NeoPixel ako indikátor stavu pre diagnostiku počas vývoja aj prevádzky.

- **Modrá**, prebiehajúca inicializácia zariadenia.
- **Oranžová**, strata spojenia s Wi-Fi alebo Azure IoT Hubom.
- **Zelená po dobu troch sekúnd**, prijatie metódy `unlock`.
- **Trojité bliknutie červenou**, prijatie metódy `deny`.
- **Vypnutá**, nečinný stav, zariadenie je pripravené prijímať volania.

3.3.3 Bezpečnostné obmedzenia prototypu

Navrhnutý bezpečnostný model definuje päť ochranných vrstiev. Implementácia prototypu má dve explicitné zjednodušenia oproti produkčnému nasadeniu, obe lokalizované vo firmvéri ESP32 zariadenia.

Chýbajúca ochrana integrity firmvéru

ESP32 v aktuálnej konfigurácii neimplementuje ochranu proti modifikácii firmvéru. Potenciálny útočník pripojený cez USB dokáže prečítať pamäť, pozmeniť ju a nahradiť naspäť, čo vytvára okno pre extrakciu uložených tokenov.

Espressif ponúka dva mechanizmy proti tomuto útoku. **Secure Boot V2** je kryptograficky podpísaný firmvér, ktorý bootloader overí proti verejnému kľúču v čipe pri štarte [62]. **Flash Encryption** je šifrovanie obsahu flash pamäte AES

klúčom uloženým v hardvéri, ktorý nie je možné po výrobe prečítať [63]. Aktivácia oboch mechanizmov je jednorázový úkon pri prvom flashovaní, vyžaduje však presnosť, lebo chyba v zápise môže dosku znehodnotiť.

Neoverený certifikát Azure IoT Hub

Komunikácia ESP32 zariadenia s Azure IoT Hub prebieha cez MQTT nad TLS 1.2. V aktuálnej implementácii prototypu však `WiFiClientSecure` neoveruje serverový certifikát Azure IoT Hub voči preddefinovanému koreňovému certifikátu. Toto zjednodušenie je teoreticky zraniteľné voči útoku typu *man-in-the-middle*, kde by útočník bol schopný umiestniť sa medzi zariadenie a Azure IoT Hub a mohol odpočúvať alebo modifikovať MQTT správy.

V produkčnom nasadení by sa tento nedostatok riešil priložením koreňového certifikátu Microsoft (DigiCert Global Root G2 [64]) priamo do firmvéru a jeho odovzdaním do `WiFiClientSecure` cez metódu `setCACert()`. Klient by potom pri TLS handshake overil identitu servera voči tomuto kotviacemu bodu. Pre pilotné nasadenie sa toto opatrenie odporúča aktivovať skôr ako akékoľvek ďalšie.

4 Evaluácia

Kapitola vyhodnocuje, do akej miery implementovaný prototyp napĺňa funkčné a nefunkčné požiadavky stanovené v prvej kapitole. Evaluácia sa opiera o manuálne testovanie pokrývajúce bežné aj hraničné scenáre, výkonnostné meranie odozvy odomykacieho procesu a overenie sieťovej kompatibility v troch sieťových prostrediach. Kapitola sa uzatvára štruktúrovaným vyhodnotením požiadaviek s jasným rozlíšením, čo je splnené, čo čiastočne a kde ostávajú otvorené otázky.

4.1 Testovacia stratégia

Testovanie prototypu prebiehalo manuálne na reálnych a simulovaných zariadeniach, bez použitia automatizovaných jednotkových či integračných testov. Tento prístup bol zvolený zámerne z troch dôvodov.

1. Kritické časti systému (čítanie NFC tagu, reakcia IoT zariadenia, end-to-end odomykací tok) sú závislé od fyzického hardvéru, ich automatizované testovanie by nezrkadlilo realitu.
2. Pre overenie súladu s požiadavkami je relevantné správanie systému ako celku, nie izolovaných komponentov.
3. Priame testovanie na fyzických zariadeniach odhaľuje aj problémy interakcie medzi komponentmi.

Testovacia zostava

Pre testovanie boli použité nasledujúce fyzické a virtuálne zariadenia.

- iPhone 15 Pro s verziou softvéru iOS 26.
- Samsung Galaxy A20e s verziou softvéru Android 11.
- iOS Simulátor iPhone 17 Pro Max s verziou softvéru iOS 26.
- SparkFun IoT RedBoard ESP32 + Qwiic Dynamic NFC/RFID Tag.

iPhone 15 Pro a Samsung Galaxy A20e boli použité pre všetky scenáre vyžadujúce reálnu NFC interakciu. Simulátory natívne nepodporujú fyzické NFC rozhranie a slúžili iba na testovanie obrazoviek, ktoré NFC interakciu nevyžadujú. Pre testy škálovateľnosti mobilnej aplikácie bolo navyše paralelne spustených 12 inštancií simulátorov na rôznych zariadeniach plus spomenuté dve fyzické zariadenia.

Sieťové prostredia

Pre overenie sieťovej kompatibility boli testy vykonané v troch rôznych sieťových prostrediach.

- Mobilné dáta s 4G a 5G pripojením, slúžili na overenie funkčnosti v mobilnom prostredí simulujúcim používanie mimo pripojenia na Wi-Fi v budove.

- Rýchla Wi-Fi sieť s rýchlosťou 1 Gbit/s pre download aj upload, s natívnou podporou IPv6, ktorá slúžila ako referenčné prostredie pre meranie latencie.
- Staršia Wi-Fi sieť s rýchlosťou 50 Mbit/s pre download aj upload, bez podpory IPv6. Toto bolo kľúčové prostredie pre overenie správneho fungovania Cloudflare Worker proxy.

Rozsah testovania a jeho obmedzenia

Testovanie pokrývalo funkčné scenáre, latenciu odomkania a sieťovú kompatibilitu. Mimo rozsah zostalo overenie škálovateľnosti na úrovni fyzických IoT zariadení, vzhľadom na dostupnosť iba jedného prototypu fyzického zariadenia.

4.2 Funkčné testovanie

Funkčné testovanie pokrýva deväť scenárov zvolených tak, aby overili jadro funkcionality systému v bežných podmienkach a primerané správanie v chybových a hraničných situáciách. Pre každý scenár je uvedený východiskový stav, postup, očakávaný a skutočný výsledok a stručné vyhodnotenie. Ak nie je uvedené inak, scenár bol testovaný na fyzickom zariadení iPhone 15 Pro a Samsung Galaxy A20e, v rýchlej Wi-Fi sieti s podporou IPv6.

T1, úspešné odomknutie s platným prístupom

Východiskový stav. Testovací účet typu `user` má pridelený prístupový kľúč s privilegiami odomknúť fyzický zámok, s časovou platnosťou zahŕňajúcou aktuálny čas. Používateľ je prihlásený v aplikácii, zariadenie je pripojené k Azure IoT Hub.

Postup. Používateľ otvorí domovskú obrazovku a ťukne na tlačidlo `TapToScan`. Systémový NFC dialóg, respektíve foreground dispatch, aktivuje čítanie. Používateľ priloží zadnú stranu telefónu k NFC tagu na zariadení.

Očakávaný výsledok. Aplikácia prečíta UUID zariadenia, volá Edge Function `unlock-device`, autorizácia prechádza, Azure IoT Hub doručí `unlock` metódu do zariadenia, LED sa rozsvieti zelenou na 3 sekundy, mobilná aplikácia zobrazí dialóg úspechu.

Skutočný výsledok. Zhodný s očakávaním. Celkový čas od priloženia telefónu po zelenú LED je rádovo 1,5 s. Scenár bol opakovane úspešne prevedený na oboch fyzických mobilných zariadeniach. Základná funkcionality systému je splnená.

T2, zamietnutie pri chýbajúcom priradení prístupu

Východiskový stav. Testovací účet bez akéhokoľvek prideleného prístupového kľúča, alebo s kľúčom bez privilegií odomknúť fyzický zámok.

Postup. Používateľ priloží telefón k NFC tagu.

Očakávaný výsledok. Edge Function `unlock-device` volá RPC metódu `can_unlock_device`, ktorá vráti `false`. Funkcia odpovedá HTTP 403. Mobilná aplikácia zobrazí dialóg zamietnutia. Azure IoT Hub dostane metódu `deny` a rozblíka LED trikrát na červeno.

Skutočný výsledok. Zhodný s očakávaním. Dialóg zamietnutia sa zobrazuje s jasnou lokalizovanou správou. LED zabliká na červeno. Negatívna autorizačná cesta funguje korektne.

T3, zamietnutie mimo časového okna platnosti

Východiskový stav. Testovací účet má pridelený prístupový kľúč obsahujúci zariadenie, avšak `valid_to_utc` je nastavený na čas v minulosti.

Postup. Používateľ priloží telefón k NFC tagu.

Očakávaný výsledok. RPC `can_unlock_device` filtruje pridelenie z dôvodu prekročenia a odpovedá aplikácii HTTP 403. Aplikácia zobrazuje dialóg a LED na zámku sa rozblíka trikrát na červeno.

Skutočný výsledok. Zhodný s očakávaním. Scenár bol rovnako otestovaný aj s `valid_to_utc` nastaveným v budúcnosti s identickým výsledkom. Časová kontrola platnosti funguje korektne v oboch smeroch.

T4, zamietnutie po predčasnom odňatí prístupu

Východiskový stav. Testovací účet má platný prístupový kľúč. Správca v samostatnej relácii cez administratívnu obrazovku označí pridelenie ako neplatné.

Postup. Krátko po zneplatnení používateľ priloží telefón k NFC tagu.

Očakávaný výsledok. RPC `can_unlock_device` deteguje neplatnosť a reaguje HTTP 403. Aplikácia zobrazuje dialóg a LED na zámku sa rozblíka trikrát na červeno.

Skutočný výsledok. Zhodný s očakávaním. Zneplatnenie prístupu sa prejavilo okamžite a žiadny cache mechanizmus účinnosť zmeny neovplyvnil. Požiadavka na okamžité odobranie prístupu úspešne splnená.

T5, zlyhanie pri chýbajúcom sieťovom pripojení telefónu

Východiskový stav. Testovací účet má pridelený prístupový kľúč s privilégiami odomknúť fyzický zámok, s časovou platnosťou zahŕňajúcou aktuálny čas. Používateľ je prihlásený v aplikácii, zariadenie je pripojené k Azure IoT Hub. Zariadenie nemá prístup k Wi-Fi ani mobilným dátam.

Postup. Používateľ priloží telefón k NFC tagu.

Očakávaný výsledok. NFC skenovanie prebehlo úspešne, volanie Edge Function však zlyhá na sieťovej chybe. Aplikácia zobrazí chybovú hlášku informujúcu o nedostupnosti pripojenia.

Skutočný výsledok. Zhodný s očakávaním. Chybový dialóg sa objaví po približne 10s s lokalizovanou správou. Používateľ má možnosť pokus opakovať. Sieťové zlyhanie je ošetrené, používateľ dostáva jasnú spätnú väzbu.

T6, zlyhanie pri chýbajúcom sieťovom pripojení IoT zariadenia

Východiskový stav. Zámok je fyzicky napájaný, avšak Wi-Fi spojenie je dočasne prerušené. Zariadenie signalizuje stratu spojenia s Azure IoT Hub prostredníctvom oranžovej LED.

Postup. Používateľ priloží telefón k NFC tagu.

Očakávaný výsledok. NFC čítanie prebehne úspešne. Edge Function autorizuje pokus. Azure IoT Hub sa pokúsi doručiť metódu `unlock` a po vypršaní

timeoutu vráti Edge Function chybu. Mobilná aplikácia zobrazí dialóg s informáciou, že zariadenie nie je dostupné.

Skutočný výsledok. Zhodný s očakávaním. Chybová správa rozlišuje medzi sieťovou chybou klienta a nedostupnosťou zariadenia. Používateľ tak vie, či má skontrolovať svoje pripojenie, alebo je chyba na strane zariadenia.

T7, zrušenie NFC skenovania používateľom

Východiskový stav. Bežná prevádzka, používateľ otvorí TapToScan widget.

Postup. Po zobrazení NFC dialógu používateľ skenovanie manuálne zruší.

Očakávaný výsledok. Natívny plugin vráti do Dart vrstvy chybový kód, aplikácia túto chybu ošetrí a vráti sa do pôvodného stavu domovskej obrazovky bez zobrazenia chybového dialógu.

Skutočný výsledok. Zhodný s očakávaním. Používateľ môže bezprostredne iniciovať ďalší pokus. Žiadna sieťová požiadavka nebola odoslaná. Aplikácia korektne rozlišuje medzi zrušením a skutočnými chybami.

T8, obnova relácie

Východiskový stav. Používateľ je prihlásený v aplikácii. Aplikácia sa ukončí a po čase znovu otvorí.

Postup. Používateľ otvorí aplikáciu. Počas štartu Supabase klient číta uloženú reláciu a podľa potreby obnovuje token.

Očakávaný výsledok. Aplikácia sa otvorí priamo na domovskej obrazovke bez potreby opätovného prihlásenia.

Skutočný výsledok. Zhodný s očakávaním. Obnova prebehne na pozadí a používateľ nevidí žiadne prechodové obrazovky ani indikátor načítavania autentifikácie. Manažment životného cyklu autentifikačnej relácie v prípade obnovy funguje korektne.

T9, vypršanie autentifikačného tokenu

Východiskový stav. Používateľ je prihlásený v aplikácii. Aplikácia sa ukončí a po čase znovu otvorí.

Postup. Pre simuláciu expirovaného refresh tokenu bola v Supabase administrácii manuálne zrušená používateľská relácia. Následne používateľ otvoril aplikáciu.

Očakávaný výsledok. Supabase klient pri pokuse o obnovu tokenu dostane chybu. Aplikácia sa prepne do neautentifikovaného stavu a navigácia presmeruje používateľa na prihlasovaciu obrazovku.

Skutočný výsledok. Zhodný s očakávaním. Presmerovanie prebehlo plynulo bez zobrazenia chybovej hlášky. Aplikácia používateľa požiadala o opätovné prihlásenie, čo je z hľadiska používateľského zážitku lepší scenár ako chybový dialóg. Manažment životného cyklu autentifikačnej relácie funguje korektne aj v prípade vypršania platnosti tokenu.

Súhrn funkčného testovania

Všetkých deväť scenárov prebehlo podľa očakávaní, čím je preukázané, že základná funkcionálna systém (od autentifikácie cez autorizáciu až po fyzickú

interakciu so zámkom) funguje korektne v bežných podmienkach aj v relevantných chybových stavoch.

Nasledujúca sekcia sa zameriava na výkonnostné charakteristiky odomykacieho toku.

4.3 Výkonnostné meranie

Odomykací tok je najkritickejší scenár systému z pohľadu používateľského zážitku, je to operácia, ktorú používateľ vykonáva niekoľkokrát do dňa a jej odozva priamo určuje, či systém pôsobí dojemom moderného *tap-to-unlock* riešenia, alebo jeho odozva je pomalšou alternatívou k mechanickému kľúču. Táto sekcia overuje splnenie nefunkčnej požiadavky s maximálnou časovou odozvou.

Metodika merania

Meranie latencie bolo realizované v réžii mobilnej aplikácie, ktorá v rámci vývojového flavoru zapisuje do aplikačného logu časové značky v kľúčových bodoch odomykacieho toku. Konkrétne sú zaznamenávané štyri body.

1. t_0 , okamih keď používateľ ťukne na **TapToScan** a aplikácia volá natívny plugin pre NFC skenovanie.
2. t_1 , okamih keď natívny plugin vráti UUID prečítaného zariadenia.
3. t_2 , okamih keď aplikácia odošle volanie Edge Function **unlock-device**.
4. t_3 , okamih keď aplikácia prijme HTTP odpoveď z Edge Function.

Z týchto bodov je možné vypočítať rozklad celkovej latencie na tri fázy.

- **Prvá fáza.** NFC skenovanie je definované rozdielom t_1 a t_0 , ktoré závisí predovšetkým od rýchlosti priloženia telefónu k tagu a od samotného NFC hardvéru.
- **Druhá fáza.** Lokálne spracovanie je definované rozdielom t_2 a t_1 , ktoré závisí od Riverpod réžie aplikácie, konverzie dát a serializácie požiadavky.
- **Tretia fáza.** Sieťová cesta a komunikácia backendu je definovaná rozdielom t_3 a t_2 a zahŕňa TLS handshake, cestu do Supabase, vyhodnotenie RPC funkcie, generovanie SAS tokenu, volanie Azure IoT Hub direct method, čakanie na odpoveď zariadenia a cestu naspäť.

Celková latencia je rozdiel t_3 a t_0 , čo je čas, ktorý používateľ reálne vníma medzi priložením telefónu a zobrazením výsledkového dialógu.

Meranie prebehlo v rýchlej Wi-Fi sieti s podporou IPv6 a spojením so Supabase cez Cloudflare proxy, čo reprezentuje najlepšie prevádzkové podmienky systému. Testovaný bol prvý scenár z predošlej sekcie. Pre každú platformu bolo zrealizovaných 30 nezávislých pokusov, pričom medzi jednotlivými pokusmi bola pauza aspoň 10 sekúnd, aby sa eliminoval vplyv cachovania TLS spojenia.

Namerané výsledky

Nasledujúce tabuľky zhrňajú štatistické charakteristiky celkovej latencie a jej rozklad po fázach. Všetky hodnoty sú uvedené v sekundách.

Platforma	Min	Medián	Priemer	Max	σ^a
iPhone 15 Pro	1,12	1,48	1,53	2,21	0,27
Samsung Galaxy A20e	1,38	1,76	1,82	2,58	0,31

Pozn: ^a Smerodajná odchýlka, všetky hodnoty v sekundách, $n = 30$ pokusov na platformu.

Tabuľka 4.1 Celková latencia odomykania, čas od priloženia telefónu po zobrazenie výsledku.

Fáza	iPhone 15 Pro	Samsung Galaxy A20e
NFC skenovanie	0,52	0,75
Lokálne spracovanie	0,04	0,06
Sieťová cesta a backend	0,97	1,01
Celkom	1,53	1,82

Pozn: Priemerné hodnoty v sekundách, $n = 30$ pokusov na platformu.

Tabuľka 4.2 Rozklad priemernej latencie odomykania na jednotlivé fázy.

Interpretácia výsledkov

Porovnanie s nefunkčnými požiadavkami. Priemerná celková latencia 1,53 s na iOS, 1,82 s na Android a maximálna nameraná hodnota 2,58 s sú pohodlne pod stanovenou hranicou troch sekúnd (NFP3). Klientský timeout nebol počas merania ani raz dosiahnutý. Nefunkčná požiadavka je splnená s rezervou, čo poskytuje priestor pre funkčnosť aj pri zhoršenej kvalite siete.

Rozklad po fázach. Hlavný príspevok k latencii tvorí tretia fáza s priemerným podielom približne 60 % celkovej latencie. Druhým významným prispievateľom je prvá fáza, ktorej čas je do značnej miery závislý od presnosti priloženia telefónu na tag a reakčného času hardvéru. Android v tejto fáze vykazuje systematicky vyššie časy, čo je v súlade s pozorovaním, že Samsung Galaxy A20e má pomalší a starší NFC čip než iPhone 15 Pro. Druhá fáza je zanedbateľná, lokálne spracovanie v aplikácii trvá rádovo desiatky milisekúnd.

Rozdiel medzi platformami. Android verzia je v priemere o 290 ms pomalšia než iOS, pričom rozdiel sa koncentruje najmä v prvej fáze. Tento rozdiel je pripísateľný hlavne rozdielu v hardvérovom výkone a mechanickému rozdielu v aktivácii NFC, kde iOS zobrazuje systémový modál okamžite, zatiaľ čo Android foreground dispatch pracuje v tichom pozadí. V oboch prípadoch ide o normálne správanie a nie o chybu.

Testovanie paralelnej prevádzky mobilnej aplikácie

Pre overenie škálovateľnosti klientskej aplikácie, najmä pre potvrdenie, že Cloudflare Worker proxy a backendová vrstva zvládajú paralelné požiadavky,

bola zrealizovaná simulácia 12 paralelných inštancií mobilnej aplikácie ako v iOS simulátore, tak v Android simulátore. Každá inštancia bola prihlásená ako odlišný testovací používateľ s vlastným prideleným prístupovým kľúčom a každá vykonávala operácie nad aplikačnými dátami.

Postup testu. Všetkých 12 simulátorov bolo spustených v priebehu približne 30 sekúnd. Následne boli na simulátoroch vykonávané rôzne používateľské interakcie.

Výsledky. Všetky požiadavky boli úspešne spracované backendom. Priemerná latencia sa pohybovala rádovo v podobných hodnotách s rozdielom maximálne 400 ms, čo je stále v rámci nefunkčných požiadaviek. Backend pri tejto mierne zvýšenej záťaži nevykazoval žiadne chyby a všetky požiadavky dostali HTTP odpoveď.

Obmedzenie merania. Test simuluje paralelnú záťaž iba na úrovni klientských požiadaviek. Keďže je dostupné jedno fyzické zariadenie, simulovaná záťaž sa zbíha na jednom Azure IoT Hub endpointe. V reálnom nasadení s desiatkami zámkov by sa paralelné požiadavky rozdelili medzi viacero nezávislých zariadení, čo by efektívnu záťaž na jednotlivé MQTT spojenia znížilo. Overenie distribuovanej záťaže by si vyžadovalo pilotné nasadenie viacerých zariadení.

Zhrnutie

Výkonnostné meranie potvrdilo, že implementácia spĺňa s rezervou nefunkčnú požiadavku v bežnej prevádzke aj v testovanej paralelnej záťaži pre klientskú vrstvu. Architektonický návrh z druhej kapitoly sa ukázal ako realistický. Overenie škálovateľnosti na úrovni fyzickej IoT infraštruktúry zostáva otvorenou otázkou pre pilotné nasadenie.

4.4 Testovanie sieťovej kompatibility

Architektonické rozhodnutie zavedenia Cloudflare Worker proxy ako riešenie IPv4 problému pre Supabase, vzniklo na základe analýzy problému. Táto sekcia overuje, že rozhodnutie bolo správne a že prepínač skutočne rieši problém s IPv6 dostupnosťou Supabase endpointu v reálnych sieťach, ktoré túto podporu nemajú.

Testované prostredia

Pre overenie sieťovej kompatibility boli zvolené tri prostredia s odlišnými charakteristikami.

- **Sieť α .** Rýchla Wi-Fi s rýchlosťou 1 Gbit/s sťahovania aj nahrávania s natívnou podporou IPv6.
- **Sieť β .** Staršia Wi-Fi s rýchlosťou 50 Mbit/s sťahovania aj nahrávania s bez podpory IPv6.
- **Sieť γ .** Mobilné dáta s premenlivou rýchlosťou sťahovania aj nahrávania s podporou IPv6.

Dostupnosť IPv6 na jednotlivých sieťach bola overená aj pomocou verejne dostupného nástroja `test-ipv6.com` priamo z prehliadača telefónu. Sieť α a γ dosiahli najlepšie výsledky, zatiaľ čo sieť β vykazuje výhradnú dostupnosť IPv4 bez IPv6.

Testovacie scenáre

Pre každú sieť boli prevedené dva scenáre, líšiace sa hodnotou prepínača `USE_CLOUDFLARE`.

Prvý scenár, priame pripojenie na Supabase. Aplikácia smeruje volania priamo na Supabase URL. V sieťach bez IPv6 by komunikácia mala zlyhať.

Druhý scenár, pripojenie cez Cloudflare Worker proxy. Aplikácia smeruje volania na Cloudflare Worker URL, ktorý ich preposiela do Supabase. Worker komunikuje cez IPv4 a IPv6 a so Supabase cez IPv6. Scenár by mal byť funkčný vo všetkých sieťach.

V každej kombinácii bol zrealizovaný prvý scenár z predchádzajúcej sekcie. Pre každú kombináciu bolo prevedených 10 nezávislých pokusov, pričom sa sledoval výsledok a priemerná latencia.

Sieť	IPv6	USE_CLOUDFLARE	Výsledok	Latencia ^a
α	OK	false	úspech	1,49
α	OK	true	úspech	1,58
β	—	false	neúspech	—
β	—	true	úspech	1,87
γ	OK	false	úspech	1,94
γ	OK	true	úspech	2,03

Pozn.: ^a Priemerná latencia v sekundách, $n = 10$ pokusov na konfiguráciu.

Tabuľka 4.3 Výsledky testov sieťovej kompatibility v troch sieťových prostrediach.

Interpretácia výsledkov

Potvrdenie problému v sieti β . V sieti bez IPv6 pri priamom pripojení na Supabase aplikácia opakovane zlyhávala s chybou pripojenia. Chyba sa prejavila ako klientský timeout. Z pohľadu používateľa bola skúsenosť neakceptovateľná a aplikácia by bola nepoužiteľná.

Úspech sieti β s prepínačom. S aktivovanou proxy vrstvou aplikácia v identickej sieti fungovala bez problémov s latenciou 1,87 s, mierne zvýšenou oproti sieti α . Funkcionálne používateľ rozdiel nepocíti.

Latencia v sieti α . Porovnanie ukazuje, že preskočenie cez Cloudflare pridáva pár milisekúnd k celkovej latencii. Toto je reálna cena za univerzálnu kompatibilitu.

Mobilné dáta. Latencia v sieti γ je systematicky o niekoľko stoviek milisekúnd vyššia než na rýchlej Wi-Fi, čo je v súlade s typickými latenciami mobilných sietí. Tieto siete operátorov majú IPv6 natívne podporovaný, takže priamy režim tu funguje rovnako spoľahlivo ako režim s proxy.

Implikácie pre nasadenie

Výsledky potvrdzujú opodstatnenosť rozhodnutia zaviesť Cloudflare Worker proxy. Pre produkčné nasadenie je odporúčané nastaviť `USE_CLOUDFLARE` na `true` ako predvolenú hodnotu, zabezpečí sa funkčnosť vo všetkých sieťových prostrediach za cenu zanedbateľnej latencie. Prepínač sa zachová pre vývoj a diagnostiku.

4.5 Vyhodnotenie funkčných požiadaviek

Tabuľka 4.4 sumarizuje stav každej z desiatich funkčných požiadaviek špecifikovaných v prvej kapitole. Stĺpec *Dôkaz* odkazuje na konkrétny testovací scenár alebo administratívnu obrazovku, kde bolo splnenie overené.

ID	Požiadavka	Stav	Dôkaz
FP1	Autentifikácia používateľa	Splnené	Scenár T8, automatická obnova relácie
FP2	Hierarchická organizačná štruktúra	Splnené	Vzorová organizácia (4 oblasti, 8 budov, 32 zariadení), CRUD overené na všetkých úrovniach
FP3	Správa používateľských účtov	Splnené	Edge Function <code>create-new-account</code> , účty pre všetky roly overené
FP4	Správa prístupových kľúčov	Splnené	Administratívna obrazovka <code>/accesses</code> , viacero kľúčov pre každú budovu
FP5	RBAC oprávnenia	Splnené	Scenáre T1 a T3
FP6	Odomknutie cez NFC	Splnené	Scenár T1, výkonnostné meranie
FP7	Centralizovaná autorizácia	Splnené	Architektúra Edge Function + RPC funkcie
FP8	Logické vymazanie	Splnené	Stĺpec <code>is_deleted</code> naprieč tabuľkami, overené mazaním na všetkých úrovniach
FP9	Viacjazyčné rozhranie	Splnené	Tri jazyky cez Flutter intl, kompletnosť prekladov overená
FP10	Vývojové a produkčné prostredie	Splnené	Dva <code>build flavors</code> , overené zostavením oboch prostredí

Tabuľka 4.4 Vyhodnotenie funkčných požiadaviek

Všetkých desať funkčných požiadaviek je v aktuálnej verzii prototypu splnených.

4.6 Vyhodnotenie nefunkčných požiadaviek

Na rozdiel od funkčných požiadaviek si nefunkčné požiadavky vyžadujú aj kvalitatívne alebo kvantitatívne argumentačné vyhodnotenie. Tabuľka 4.5 zhŕňa stav, pričom pre dve požiadavky, NFP3 a NFP4, je potrebná podrobnejšia argumentácia.

ID	Požiadavka	Stav	Dôkaz
NFP1	Multiplatformnosť	Splnené	T1 až T9 prebehli identicky na iPhone 15 Pro a Samsung Galaxy A20e
NFP2	Bezpečnosť komunikácie a údajov	Čiastočne splnené	TLS, JWT v SecureStorageService, obmedzenie firmvéru (sekcia 3.3.3)
NFP3	Čas odozvy	Splnené s rezervou	Sekcia 4.3, priemerná latencia 1,53 s až 1,82 s
NFP4	Škálovateľnosť	Čiastočne splnené	Architektonicky podporená, fyzicky neoverená pre IoT vrstvu
NFP5	Použiteľnosť	Splnené	Hlavný tok jedným ťuknutím, oddelená administratívna sekcia
NFP6	Udržiavateľnosť	Splnené	Oddelenie vrstiev v zdrojovom kóde mobilnej aplikácie
NFP7	Sieťová kompatibilita	Splnené	Sekcia 4.4, overené v sieti bez podpory IPv6

Tabuľka 4.5 Vyhodnotenie nefunkčných požiadaviek

NFP3 (čas odozvy). Meranie ukázalo priemernú latenciu 1,53 s na iOS a 1,82 s na Android s maximom 2,58 s naprieč všetkými nezávislými pokusmi v bežných podmienkach. Tieto hodnoty sú pohodlne pod cieľom troch sekúnd. Aj v paralelnej záťaži 12 simulátorov maximálna latencia nepresiahla 3 s.

NFP4 (škálovateľnosť). Architektúra túto požiadavku napĺňa, Supabase je postavený na PostgreSQL s podporou horizontálneho škálovania cez read-replicas, Edge Functions sú stateless a škálujú automaticky podľa záťaže, Azure IoT Hub má deklarované kvóty na milióny paralelných MQTT spojení [65] a Cloudflare Workers bežia v globálnej edge sieti. Architektúra teda škálovaniu nebráni. Overenie produkčnej škálovateľnosti na úrovni fyzických zariadení by si však vyžadovalo pilotné nasadenie.

4.7 Súhrn evaluácie a výhľad

Funkčné testovanie deviatich scenárov, výkonnostné meranie a testovanie sieťovej kompatibility potvrdili, že systém funguje podľa návrhu. Z pohľadu deklarovaných požiadaviek systém napĺňa všetkých desať funkčných a päť zo siedmich nefunkčných požiadaviek plne, dve nefunkčné požiadavky (NFP2 a NFP4) sú splnené čiastočne. Najdôležitejším výsledkom je, že autorizačné rozhodnutie zostáva výhradne v doméne serverovej vrstvy, čo je kľúčová bezpečnostná vlastnosť navrhutej architektúry.

Evaluácia identifikovala dva otvorené body, ktoré nepredstavujú nedostatky v zmysle nesplnených funkčných cieľov, ale smerovania pre ďalšie iterácie.

Overenie produkčnej škálovateľnosti pilotným nasadením s desiatkami fyzických zariadení v skutočnej budove a **sprísnenie bezpečnostných opatrení v ESP32 firmvéri** cez Secure Boot V2, Flash Encryption a overenie certifikátu Azure IoT Hub.

Identifikované obmedzenia nie sú dôsledkom architektonických chýb, ale typických kompromisov prototypovej fázy. Sú lokalizované a riešiteľné v rámci existujúcej architektúry bez nutnosti zásadného prepracovania.

Záver

Cieľom tejto bakalárskej práce bolo navrhnúť a implementovať cross-platformovú mobilnú aplikáciu pre kontrolu prístupu do budov ako modernú alternatívu k tradičným mechanickým kľúčom a kartovým systémom. Práca v tomto smere dosiahla funkčný prototyp pokrývajúci všetky vrstvy systému a overila splnenie stanovených požiadaviek.

V analytickej časti bola vykonaná rešerš existujúcich riešení a iteratívne hľadanie vhodnej NFC architektúry, ktoré viedlo cez tri prototypy k finálnemu riešeniu s pasívnym dynamickým NFC tagom čítaným telefónom. V návrhovej časti bola predstavená päť-komponentová architektúra spolu s doménovo špecifickým dátovým modelom postaveným na abstrakcii prístupového kľúča s časovou platnosťou na úrovni pridelenia. Implementačná časť pokrýva všetky vrstvy systému, od mobilnej aplikácie vo Flutteri s natívnymi NFC pluginmi pre iOS a Android, cez backendovú vrstvu kombinujúcu Supabase a Azure IoT Hub, až po firmvér ESP32 zariadenia. Bezpečnostný model je koncipovaný ako *defense in depth* na piatich nezávislých vrstvách.

Evaluácia potvrdila funkčnú správnosť systému deviatimi testovacími scenármi, výkonnostným meraním latencie odomykania s priemernou hodnotou 1,5s až 1,8s a overením sieťovej kompatibility. Zo všetkých funkčných požiadaviek je splnených desať z desiatich, zo siedmich nefunkčných je päť plne a dve čiastočne. Neoverená zostáva škálovateľnosť na úrovni fyzickej IoT infraštruktúry, ktorá si vyžaduje pilotné nasadenie viacerých zariadení.

Pre prechod od prototypu k produkčne nasaditeľnému systému sú navrhnuté konkrétne smery ďalšieho vývoja. Patrí sem pilotné nasadenie s desiatkami zariadení, sprísnenie bezpečnostných opatrení v ESP32 firmvéri a v dlhodobom horizonte integrácia s natívnymi mobilnými peňaženkami. Identifikované obmedzenia prototypu sú explicitne dokumentované spolu s konkrétnymi cestami k ich riešeniu, pričom žiadne z nich nie je dôsledkom architektonických chýb, ale typickým kompromisom prototypovej fázy, riešiteľnými v rámci existujúceho návrhu.

Výsledkom práce je funkčný prototyp, ktorý naplňa stanovené ciele a poskytuje solídnu základňu pre ďalšie iterácie smerom k produkčnému nasadeniu. Práca tým ukazuje realizovateľnosť otvoreného alternatívneho stacku k uzavretým komerčným platformám.

Literatúra

1. EUROSTAT. *Digital economy and society statistics, household and individuals* [online]. [cit. 2026-04-10]. Dostupné z : https://ec.europa.eu/eurostat/statistics-explained/index.php?title=Digital_economy_and_society_statistics_-_households_and_individuals.
2. KONING GANS, Gerhard de; HOEPMAN, Jaap-Henk; GARCIA, Flavio D. A Practical Attack on the MIFARE Classic. In: *Smart Card Research and Advanced Applications, CARDIS 2008*. Springer, 2008. Dostupné z DOI: 10.1007/978-3-540-85893-5_20.
3. COURTOIS, Nicolas T. The Dark Side of Security by Obscurity and Cloning MiFare Classic Rail and Building Passes, Anywhere, Anytime. In: *Proceedings of the International Conference on Security and Cryptography (SECRYPT 2009)*. SciTePress, 2009. ISBN 978-989-674-005-4.
4. NXP SEMICONDUCTORS. *MIFARE DESFire EV2 2K/8K, Product overview* [online]. [cit. 2026-04-10]. Dostupné z : https://www.nxp.com/products/MIFARE_DESFIRE_EV2_2K_8K.
5. HID GLOBAL. *MIFARE DESFire EV3 and iCLASS solutions for mobile access* [online]. [cit. 2026-04-10]. Dostupné z : <https://www.hidglobal.com/products/mifare-desfire-ev3-iclass>.
6. SALTO SYSTEMS. *Digital Key, your phone as a tag* [online]. [cit. 2026-04-10]. Dostupné z : <https://saltosystems.com/en/blog/digital-key-your-phone-as-a-tag-2/>.
7. STAAT, Paul; JANSEN, Kai; ZENGER, Christian; ELDERS-BOLL, Harald; PAAR, Christof. Analog Physical-Layer Relay Attacks with Application to Bluetooth and Phase-Based Ranging. In: *Proceedings of the 15th ACM Conference on Security and Privacy in Wireless and Mobile Networks (WiSec '22)*. ACM, 2022. Dostupné z DOI: 10.1145/3507657.3528536.
8. SALTO SYSTEMS. *NFC in Salto KS Keychain* [online]. [cit. 2026-04-10]. Dostupné z : <https://support.saltosystems.com/ks/mobile/saltoks-keychain-app/nfc/>.
9. APPLE INC. *Access key types, Apple Platform Security* [online]. [cit. 2026-04-10] Dostupné z : <https://support.apple.com/sk-sk/guide/security/sec30bdef041/web>.
10. APPLE INC. *Apple Platform Security* [online]. [cit. 2026-04-10]. Dostupné z : <https://support.apple.com/guide/security/welcome/web>.
11. APPLE INC. *Apple Wallet, developer overview and keys for partners* [online]. [cit. 2026-04-10]. Dostupné z : <https://developer.apple.com/wallet/>. Súvisiaci odkaz: <https://learn.wallet.apple/keys>.
12. INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. *ISO/IEC/IEEE 29148:2018, Systems and software engineering, Life cycle processes, Requirements engineering* [online]. [cit. 2026-04-10]. Dostupné z : <https://www.iso.org/standard/72089.html>.

13. RESCORLA, Eric. *The Transport Layer Security (TLS) Protocol Version 1.3* [online]. [cit. 2026-04-10]. Dostupné z : <https://www.rfc-editor.org/rfc/rfc8446>.
14. JONES, Michael B.; BRADLEY, John; SAKIMURA, Nat. *JSON Web Token (JWT)* [online]. [cit. 2026-04-10]. Dostupné z : <https://www.rfc-editor.org/rfc/rfc7519>.
15. RASPBERRY PI LTD. *Raspberry Pi Zero 2 W Product Brief* [online]. [cit. 2026-04-10]. Dostupné z : <https://pip-assets.raspberrypi.com/categories/584-raspberry-pi-zero-2-w/documents/RP-008359-DS-1-raspberry-pi-zero-2-w-product-brief.pdf?disposition=inline>.
16. PiCOCKPIT. *Everything about Raspberry Pi Zero 2 W* [online]. [cit. 2026-04-10] Dostupné z : <https://picockpit.com/raspberry-pi/everything-about-raspberry-pi-zero-2-w/>.
17. RASPBERRY PI LTD. *Raspberry Pi Pico W Datasheet, An RP2040-based microcontroller board with wireless* [online]. [cit. 2026-04-10]. Dostupné z : <https://pip-assets.raspberrypi.com/categories/686-raspberry-pi-pico-w/documents/RP-008312-DS-1-pico-w-datasheet.pdf?disposition=inline>.
18. RASPBERRY PI LTD. *Raspberry Pi Pico 2 W Datasheet, An RP2350-based microcontroller board with wireless* [online]. [cit. 2026-04-10]. Dostupné z : <https://pip-assets.raspberrypi.com/categories/1088-raspberry-pi-pico-2-w/documents/RP-008304-DS-2-pico-2-w-datasheet.pdf?disposition=inline>.
19. SPARKFUN ELECTRONICS. *SparkFun IoT RedBoard, ESP32 Development Board* [online]. [cit. 2026-04-10]. Dostupné z : <https://www.sparkfun.com/sparkfun-iot-redboard-esp32-development-board.html>.
20. SPARKFUN ELECTRONICS. *IoT RedBoard ESP32 Development Board Hookup Guide* [online]. [cit. 2026-04-10]. Dostupné z : <https://learn.sparkfun.com/tutorials/iot-redboard-esp32-development-board-hookup-guide/all>.
21. PLATFORMIO. *SparkFun ESP32 IoT RedBoard, PlatformIO documentation* [online]. [cit. 2026-04-10]. Dostupné z : https://docs.platformio.org/en/latest/boards/espressif32/sparkfun_esp32_iot_redboard.html.
22. SPARKFUN ELECTRONICS. *Qwiic Dynamic NFC/RFID Tag Hookup Guide* [online]. [cit. 2026-04-10]. Dostupné z : <https://learn.sparkfun.com/tutorials/qwiic-dynamic-nfc-rfid-tag-hookup-guide>.
23. GOOGLE LLC. *Flutter, Multi-platform* [online]. [cit. 2026-04-10]. Dostupné z : <https://docs.flutter.dev/platform-integration>.
24. GOOGLE LLC. *Writing custom platform-specific code, Flutter platform channels* [online]. [cit. 2026-04-10]. Dostupné z : <https://docs.flutter.dev/platform-integration/platform-channels>.
25. SUPABASE. *Supabase, The Open Source Firebase Alternative* [online]. [cit. 2026-04-10]. Dostupné z : <https://supabase.com/docs>.

26. MICROSOFT CORPORATION. *IoT concepts and Azure IoT Hub* [online]. [cit. 2026-04-10]. Dostupné z : <https://learn.microsoft.com/en-us/azure/iot-hub/iot-concepts-and-iot-hub>.
27. MICROSOFT CORPORATION. *Use MQTT to communicate with Azure IoT Hub* [online]. [cit. 2026-04-10]. Dostupné z : <https://learn.microsoft.com/en-us/azure/iot-hub/iot-mqtt-connect-to-iot-hub>.
28. MICROSOFT CORPORATION. *Understand and invoke direct methods from IoT Hub* [online]. [cit. 2026-04-10]. Dostupné z : <https://learn.microsoft.com/en-us/azure/iot-hub/iot-hub-devguide-direct-methods>.
29. GOOGLE LLC. *Host-based card emulation overview, Android Developers* [online]. [cit. 2026-04-10]. Dostupné z : <https://developer.android.com/develop/connectivity/nfc/hce>.
30. APPLE INC. *Introducing Core NFC, WWDC17 Session 718* [online]. [cit. 2026-04-10]. Dostupné z : <https://developer.apple.com/videos/play/wwdc2017/718/>.
31. LOMAS, Natasha. *EU ends Apple Pay antitrust probe with binding commitments to open up contactless payments* [online]. [cit. 2026-04-10]. Dostupné z : <https://techcrunch.com/2024/07/11/eu-ends-apple-pay-antitrust-probe-with-binding-commitments-to-open-up-contactless-payments/>.
32. APPLE INC. *HCE transactions in apps, Apple Developer Documentation* [online]. [cit. 2026-04-10]. Dostupné z : <https://developer.apple.com/support/hce-transactions-in-apps/>.
33. APPLE INC. *CoreNFC, Apple developer Documentation* [online]. [cit. 2026-04-10] Dostupné z : <https://developer.apple.com/documentation/corenfc>.
34. NXP SEMICONDUCTORS. *PN532/C1 Near Field Communication (NFC) controller, Product data sheet, Rev. 3.6* [online]. [cit. 2026-04-10]. Dostupné z : https://www.nxp.com/docs/en/nxp/data-sheets/PN532_C1.pdf.
35. COSKUN, Vedat; OZDENIZCI, Busra; OK, Kerem. A Survey on Near Field Communication (NFC) Technology. *Wireless Personal Communications*. 2013. Dostupné z DOI: 10.1007/s11277-012-0935-5.
36. SEEED-STUDIO (GITHUB). *PN532 Card Emulation Problem, Issue #111* [online]. [cit. 2026-04-10]. Dostupné z : <https://github.com/Seeed-Studio/PN532/issues/111>.
37. NXP COMMUNITY. *PN532 Tag emulation NFC Tools showing NULL all the data* [online]. [cit. 2026-04-10]. Dostupné z : <https://community.nxp.com/t5/NFC/PN532-Tag-emulation-NFC-Tools-showing-NULl-all-the-data/td-p/2218033>.
38. NXP COMMUNITY. *PN532 card emulation mode issue, odporúčanie PN7160/CLRC663* [online]. [cit. 2026-04-10]. Dostupné z : <https://community.nxp.com/t5/NFC/PN532-card-emulation-mode-issue/m-p/1720851>.
39. SPARKFUN ELECTRONICS. *SparkFun Qwiic Dynamic NFC/RFID Tag (ST25DV64KC)* [online]. [cit. 2026-04-10]. Dostupné z : <https://www.sparkfun.com/sparkfun-qwiic-dynamic-nfc-rfid-tag.html>.

40. SPARKFUN ELECTRONICS. *Qwiic Connect System* [online]. [cit. 2026-04-10]. Dostupné z : <https://www.sparkfun.com/qwiic>.
41. SUPABASE. *Connection management and networking, Supabase Docs* [online]. [cit. 2026-04-10]. Dostupné z : <https://supabase.com/docs/guides/database/connection-management>.
42. SUPABASE. *IPv4 a IPv6 compatibility, Supabase Docs* [online]. [cit. 2026-04-10]. Dostupné z : <https://supabase.com/docs/guides/platform/ipv4-address>.
43. ROSS, Ron; MCEVILLE, Michael; OREN, Janet C. *Systems Security Engineering: Considerations for a Multidisciplinary Approach in the Engineering of Trustworthy Secure Systems*. 2018. Tech. spr. National Institute of Standards a Technology (NIST). Dostupné z DOI: 10.6028/NIST.SP.800-160v1.
44. SHOSTACK, Adam. *Threat Modeling: Designing for Security*. Wiley, 2014. ISBN 978-1-118-80999-0.
45. SUPABASE. *Row Level Security (RLS), Supabase Docs* [online]. [cit. 2026-04-10]. Dostupné z : <https://supabase.com/docs/guides/database/postgres/row-level-security>.
46. SALTZER, Jerome H.; SCHROEDER, Michael D. The Protection of Information in Computer Systems. *Proceedings of the IEEE*. 1975.
47. LLC, Google. *Flutter Architectural Overview* [online]. [cit. 2026-04-10]. Dostupné z : <https://docs.flutter.dev/resources/architectural-overview>.
48. NFC FORUM. *NFC Data Exchange Format (NDEF) Technical Specification* [online]. [cit. 2026-04-10]. Dostupné z : <https://nfc-forum.org/build/specifications/data-exchange-format-ndef-technical-specification/>
49. SUPABASE. *User sessions, Supabase Auth Docs* [online]. [cit. 2026-04-10]. Dostupné z : <https://supabase.com/docs/guides/auth/sessions>.
50. APPLE INC. *Keychain services, Apple Developer Documentation* [online]. [cit. 2026-04-10]. Dostupné z : <https://developer.apple.com/documentation/security/keychain-services>.
51. APPLE INC. *Protecting Keys with the Secure Enclave* [online]. [cit. 2026-04-10]. Dostupné z : <https://developer.apple.com/documentation/security/protecting-keys-with-the-secure-enclave>. Doplnkový kontext: <https://support.apple.com/sk-sk/guide/security/sec59b0b31ff/web>.
52. GOOGLE LLC. *Security, Jetpack (EncryptedSharedPreferences)* [online]. [cit. 2026-04-10]. Dostupné z : <https://developer.android.com/jetpack/androidx/releases/security>.
53. GOOGLE LLC. *Creating flavors of a Flutter app* [online]. [cit. 2026-04-10]. Dostupné z : <https://docs.flutter.dev/deployment/flavors>.
54. FLUTTER COMMUNITY. *flutter dotenv, Easily configure any Flutter application with global variables using a .env file* [online]. [cit. 2026-04-10]. Dostupné z : https://pub.dev/packages/flutter_dotenv.
55. FLUTTER TEAM. *go router, A declarative routing package for Flutter* [online]. [cit. 2026-04-10]. Dostupné z : https://pub.dev/packages/go_router.

56. GOOGLE LLC. *Internationalizing Flutter apps* [online]. [cit. 2026-04-10]. Dostupné z : <https://docs.flutter.dev/ui/internationalization>.
57. CLOUDFLARE. *Cloudflare Workers, Overview* [online]. [cit. 2026-04-10]. Dostupné z : <https://developers.cloudflare.com/workers/>.
58. SUPABASE. *Edge Functions, Supabase Docs* [online]. [cit. 2026-04-10]. Dostupné z : <https://supabase.com/docs/guides/functions>.
59. SUPABASE. *Edge Functions Architecture, Supabase Docs* [online]. [cit. 2026-04-10] Dostupné z : <https://supabase.com/docs/guides/functions/architecture>
60. SUPABASE. *Database Functions, Supabase Docs* [online]. [cit. 2026-04-10]. Dostupné z : <https://supabase.com/docs/guides/database/functions>.
61. O'LEARY, Nick. *PubSubClient, A client library for Arduino that provides support for MQTT* [online]. [cit. 2026-04-10]. Dostupné z : <https://github.com/knolleary/pubsubclient>.
62. ESPRESSIF SYSTEMS. *Secure Boot V2, ESP32 (ESP-IDF Programming Guide)* [online]. [cit. 2026-04-10]. Dostupné z : <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/security/secure-boot-v2.html>.
63. ESPRESSIF SYSTEMS. *Flash Encryption, ESP32 (ESP-IDF Programming Guide)* [online]. [cit. 2026-04-10]. Dostupné z : <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/security/flash-encryption.html>.
64. MICROSOFT CORPORATION. *Azure Certificate Authority details* [online]. [cit. 2026-04-10]. Dostupné z : <https://learn.microsoft.com/en-us/azure/security/fundamentals/azure-certificate-authority-details?tabs=root-and-subordinate-cas-list>.
65. MICROSOFT CORPORATION. *Understand Azure IoT Hub quotas and throttling* [online]. [cit. 2026-04-10]. Dostupné z : <https://learn.microsoft.com/en-us/azure/iot-hub/iot-hub-devguide-quotas-throttling>.
66. ŠUPEJ, Adam. *Access Control repository* [online]. [cit. 2026-05-05]. Dostupné z : <https://github.com/aaackoo/Access-Control>.

Zoznam obrázkov

2.1	Abstraktný pohľad na architektúru systému	23
2.2	Databázová schéma	26
2.3	Pohľad odomykania z perspektívy mobilnej aplikácie	30
2.4	Pohľad backendovej autorizačnej logiky	31
2.5	Pohľad reakcie zámku	33
3.1	Platform channels (zdroj: [47]).	38
3.2	Natívny modálny dialóg.	39
3.3	Administratívna obrazovka správy spoločnosti so záložkou budovy.	46
3.4	Domovská obrazovka aplikácie.	47

Zoznam použitých skratiek

RFID - rádiová frekvencia (Radio-Frequency Identification),
NFC - komunikácia v blízkom poli (Near Field Communication),
BaaS - backend ako služba (Backend as a Service),
IoT - internet vecí (Internet of Things),
GCM - Galoisov/počítadlový režim blokovej šifry (Galois/Counter Mode),
MAC - autentifikačný kód správy (Message Authentication Code),
SHA - rodina kryptografických hashovacích funkcií (Secure Hash Algorithm),
EAL - úroveň záruky bezpečnostnej certifikácie (Evaluation Assurance Level),
AES - symetrický šifrovací štandard (Advanced Encryption Standard),
HMAC - autentifikačný kód správy založený na hashovacej funkcii (Hash-based Message Authentication Code),
BLE - energeticky úsporný Bluetooth (Bluetooth Low Energy),
RBAC - riadenie prístupu na základe rolí (Role-Based Access Control),
FP - funkčná požiadavka,
NFP - nefunkčná požiadavka,
TLS - protokol pre šifrovanú transportnú vrstvu (Transport Layer Security),
JWT - webový autentifikačný token vo formáte JSON (JSON Web Token),
JSON - formát na výmenu štruktúrovaných dát (JavaScript Object Notation),
URI - jednotný identifikátor zdroja (Uniform Resource Identifier),
HTTP - protokol pre prenos hypertextu (Hypertext Transfer Protocol),
HTTPS - HTTP zabezpečené cez TLS (Hypertext Transfer Protocol Secure),
REST - architektonický štýl webových služieb (Representational State Transfer),
RPC - vzdialené volanie procedúr (Remote Procedure Call),
MQTT - protokol pre publish-subscribe komunikáciu IoT zariadení (Message Queuing Telemetry Transport),
AMQP - protokol pre zasielanie správ (Advanced Message Queuing Protocol),
EEPROM - elektricky vymazateľná programovateľná pamäť (Electrically Erasable Programmable Read-Only Memory),
ROM - pamäť určená iba na čítanie (Read-Only Memory),
PCB - plošný spoj (Printed Circuit Board),
I2C - sériová zbernica pre prepojenie integrovaných obvodov (Inter-Integrated Circuit),
RF - rádiová frekvencia (Radio Frequency),
SPI - sériové periférne rozhranie (Serial Peripheral Interface),
NDEF - formát výmeny dát NFC tagov (NFC Data Exchange Format),
UUID - univerzálne unikátny identifikátor (Universally Unique Identifier),
UI - používateľské rozhranie (User Interface),
AoT - kompilácia pred spustením programu (Ahead-of-Time),
API - rozhranie pre programovanie aplikácií (Application Programming Interface),
SDK - sada nástrojov pre vývoj softvéru (Software Development Kit),
RLS - zabezpečenie na úrovni riadkov v databáze (Row-Level Security),
CRUD - databázové operácie vytvorenia, čítania, úpravy a odstránenia (Create, Read, Update, Delete),

NAT - prekladanie sieťových adries (Network Address Translation),
SAS - podpisová schéma pre časovo obmedzený prístup (Shared Access Signature),
HCE - softvérová emulácia bezkontaktnéj karty (Host Card Emulation),

A Prílohy

Zdrojový kód všetkých komponentov systému je dostupný v repozitári [66] priloženom k tejto práci. Repozitár je organizovaný do samostatných adresárov podľa komponentov.

A.1 Mobilná aplikácia

Cross-platformová mobilná aplikácia v jazyku Dart postavená nad frameworkom Flutter, určená pre platformy iOS a Android. Súčasťou sú aj natívne pluginy pre prácu s NFC. Pre iOS v jazyku Swift nad frameworkom CoreNFC a pre Android v jazyku Kotlin nad rozhraním NfcAdapter.

Umiestnenie v repozitári. `mobile_app/`. Hlavná aplikačná logika je v adresári `lib/`, s natívnymi pluginmi v podadresári `ios/Runner/` a `android/app/`.

A.2 Arduino kód

Firmvér pre mikrokontrolér ESP32 v Arduino frameworku, zabezpečujúci inicializáciu dynamického NFC tagu a obojsmernú MQTT komunikáciu s Azure IoT Hub.

Umiestnenie v repozitári. `esp32_IoT/`. Hlavný vstupový bod je `iot_device.ino` a šablóna pre konfiguračné premenné je v súbore `secrets.h.example`.

A.3 Edge Functions

Serverless funkcie v jazyku TypeScript bežiace v Deno runtime na platforme Supabase. Zabezpečujú autorizáciu odomykania, vytváranie účtov a registráciu zariadení v Supabase aj Azure IoT Hub.

Umiestnenie v repozitári. `supabase/functions/`. Každá Edge Function je v samostatnom súbore s názvom identickým s ich funkciou.

A.4 Supabase schéma

Kompletná schéma databázy obsahujúca tabuľky, databázové funkcie a definíciu enumeračných typov.

Umiestnenie v repozitári. Jednoduchý súbor `supabase/supabase_dump.sql`

A.5 Cloudflare Worker

Proxy vrstva v jazyku JavaScript zabezpečujúca dostupnosť backendu zo sietí bez podpory IPv6.

Umiestnenie v repozitári. Jednoduchý súbor `cloudflare_worker/worker.js`.